Digital Food-Ordering With AWS Serverless

Sem End Project Report

Cloud and Serverless Computing

Department of computer science and Engineering

By

2200031336

Section – 32

Advanced

Under the supervision of

Dr. S.Kavitha

Koneru Lakshmaiah Education

Foundation

(Deemed to be University estd., u/s 3 of UGC Act 1956)

Green Fields, Vaddeswaram, Guntur (Dist.), Andhra Pradesh – 522302

CONTENTS

LIST OF FIGURES

Abstract

The Digital Food Ordering System using AWS Serverless Architecture streamlines online food ordering, processing, and delivery through a scalable and cost-efficient cloud-based solution. The frontend is built using a free HTML/CSS template and hosted on Amazon S3, with CloudFront enabling fast and reliable content delivery across global locations.

The backend APIs are developed using AWS Lambda in conjunction with Amazon API Gateway, adopting a fully serverless and event-driven approach that ensures automatic scaling, reduced operational overhead, and seamless integration of business logic. Core user and order data are stored using Amazon DynamoDB or Amazon RDS, depending on the specific data structure and performance requirements.

For asynchronous communication and smooth order lifecycle management, the system leverages Amazon SNS (Simple Notification Service) and Amazon SQS (Simple Queue Service), allowing real-time alerts and decoupled processing of tasks such as order confirmation and kitchen updates.

Security is a critical component, enforced using AWS WAF to guard against common web exploits, and AWS IAM roles to manage access permissions with fine-grained control, ensuring data protection and privacy across the platform.

Introduction

This project focuses on the development of a real-time, scalable, and serverless digital food ordering system using AWS cloud services. The system is designed to provide users with a seamless and efficient way to browse menus, place food orders, and receive confirmation all without the need for managing any backend servers. By leveraging the power of AWS serverless architecture, the solution ensures high availability, low maintenance, and effortless scalability.

To achieve this, core AWS services such as Amazon S3 and CloudFront are used to host and deliver a responsive web interface, while API Gateway and AWS Lambda handle backend business logic in a fully managed environment. Amazon DynamoDB is used to store real-time order details, while Amazon RDS stores structured customer and order data. Amazon SNS is integrated to send real-time email alerts about new orders, and Amazon Cognito manages user authentication securely.

The food ordering workflow begins with a customer placing an order through the web interface. The order details are processed via Lambda, saved to both DynamoDB and RDS, and a notification is sent using SNS. The system also includes Amazon CloudWatch for monitoring and logging Lambda executions, ensuring operational visibility and debugging support.

This project demonstrates how a fully serverless architecture can be used to build a modern food ordering platform that is not only cost-efficient and easily maintainable but also capable of handling growing traffic and dynamic user demands with minimal effort. The solution ensures real-time responsiveness and personalized user interaction, resulting in an enhanced digital dining experience.

Literature Review

Overview of Digital Food Ordering System

Digital food ordering systems have transformed how restaurants interact with customers by enabling online browsing, ordering, and payment. Traditional systems often faced challenges in scalability and maintenance. Serverless technologies, especially from AWS, provide an efficient solution.

AWS services like Lambda, API Gateway, DynamoDB, and SNS allow food ordering platforms to run backend logic, store order data, and send real-time notifications without managing servers. Amazon Cognito ensures secure user authentication, while CloudWatch provides monitoring and logs.

These systems improve performance, reduce costs, and offer personalized user experiences. The use of recommendation techniques, like collaborative filtering, can further enhance the service by suggesting popular or personalized food items based on user behavior.

Overall, AWS serverless technologies offer a modern, resilient framework for building and deploying digital food ordering systems that are scalable, cost-efficient, and responsive to dynamic user demands.

Project Objectives and Scope

Problem Statement:

In today's fast-paced digital world, users expect quick, personalized, and seamless food ordering experiences. Traditional food ordering systems often rely on monolithic architectures, manual data handling, or static menus that do not adapt to individual user preferences. As a result, users may struggle to find their favorite dishes, experience delays in service, or encounter scalability issues during high-demand periods.

This project aims to solve these challenges by developing a real-time, serverless digital food ordering system using AWS services. The objective is to automate the order management process—from capturing user requests and storing order data to sending real-time notifications—while ensuring high availability, low latency, and cost-efficiency. By utilizing services like AWS Lambda, API Gateway, DynamoDB, RDS, and SNS, the system is designed to scale dynamically and operate without server maintenance overhead.

Additionally, the architecture allows for future integration of personalized food recommendations using tools like Amazon Personalize, providing tailored dish suggestions based on user behavior. The goal is to enhance the overall food ordering experience through automation, personalization, and reliability—making it suitable for both startups and large-scale food delivery platforms.

Technical Implementation

5.1 Data Gathering

Data gathering plays a crucial role in building a robust digital food ordering system. The project requires structured data collection involving customer details, food menu items, pricing, and order history to enable smooth processing and potential future personalization.

- **Data Sources:** The data is collected from either sample datasets or real-time interactions from users on the food ordering platform. Key data points include:
    - **User Data**: Customer names, contact details (e.g., phone or email), location, and previous orders.
    - Food Menu Data: Item names, categories (e.g., veg, non-veg, beverages), prices, availability, and nutritional information.
    - **Order Data:** Timestamps, order ID, selected items, total price, status (e.g., pending, confirmed, delivered).
- **Data Preparation and Storage:** Data is validated, filtered, and transformed for efficient storage and retrieval. Amazon DynamoDB is used for real-time, scalable storage of order records, while Amazon RDS (MySQL) stores structured relational data like menu items and user profiles. The data pipeline ensures seamless integration and quick access through API Gateway and AWS Lambda.
- **Data Privacy and Ethics:** If the system involves actual users, it adheres to data protection standards by anonymizing sensitive details and implementing secure transmission protocols (HTTPS, IAM roles). Best practices and regulations such as GDPR are considered to protect user identity and transaction records.

5.2 Creativity & Originality

Although the core implementation utilizes AWS-managed services, the originality of the system lies in its design and user-centric approach.

- Serverless Architecture: The use of AWS Lambda for backend logic, Amazon API Gateway for request routing, DynamoDB for quick lookups, and SNS for notifications reflects a creative, cost-effective, and scalable architecture without the need for managing infrastructure.
- Order Personalization: The system can be extended creatively by integrating recommendation engines that suggest frequently ordered items or combos based on previous behavior or time of day.
- User Interface Design: The frontend interface (web or mobile) focuses on an intuitive layout that enables quick menu browsing, dynamic filters (e.g., veg/non-veg, spicy/mild), and seamless checkout. Interactive elements like live order tracking and feedback options enhance user engagement.
- **Enhanced Features:**
    - **Real-time Alerts**: SNS is used to notify users instantly when an order is placed or updated.
    - **Weather-Based Specials:** Integrating external APIs can allow the system to suggest comfort foods on rainy days or cool beverages in hot weather.
    - **Feedback Loop:** Post-order feedback is collected and used to refine offerings or identify best-performing menu items.
- **Continuous Evolution:** The architecture supports easy integration of analytics tools like Amazon QuickSight for reporting or Amazon Personalize for future recommendation features, enabling the system to grow smarter over time.

Analysis & Problem-Solving

The goal is to build a real-time, personalized digital food ordering system that not only allows users to browse and place orders seamlessly but also provides smart recommendations based on user behaviour and preferences. This involves developing a serverless architecture that supports dynamic updates, personalized suggestions, and smooth transaction processing.

Key Components Involved:

1. **User Data:**
    - Understand user profiles including order history, food preferences (e.g., veg/non-veg, spicy/mild), and location.
2. **Menu & Item Data:**
    - Information about food items—categories, ingredients, pricing, availability, and nutritional info.
3. **User-Item Interactions:**
    - Track and log user actions such as browsing items, adding to cart, past purchases, ratings, and preferences.
4. **Real-Time Personalization:**
    - The system must adapt dynamically to user behavior, offering personalized food suggestions (e.g., "Your usual", "Recommended for lunch", "Popular in your area").

Problem-Solving Process

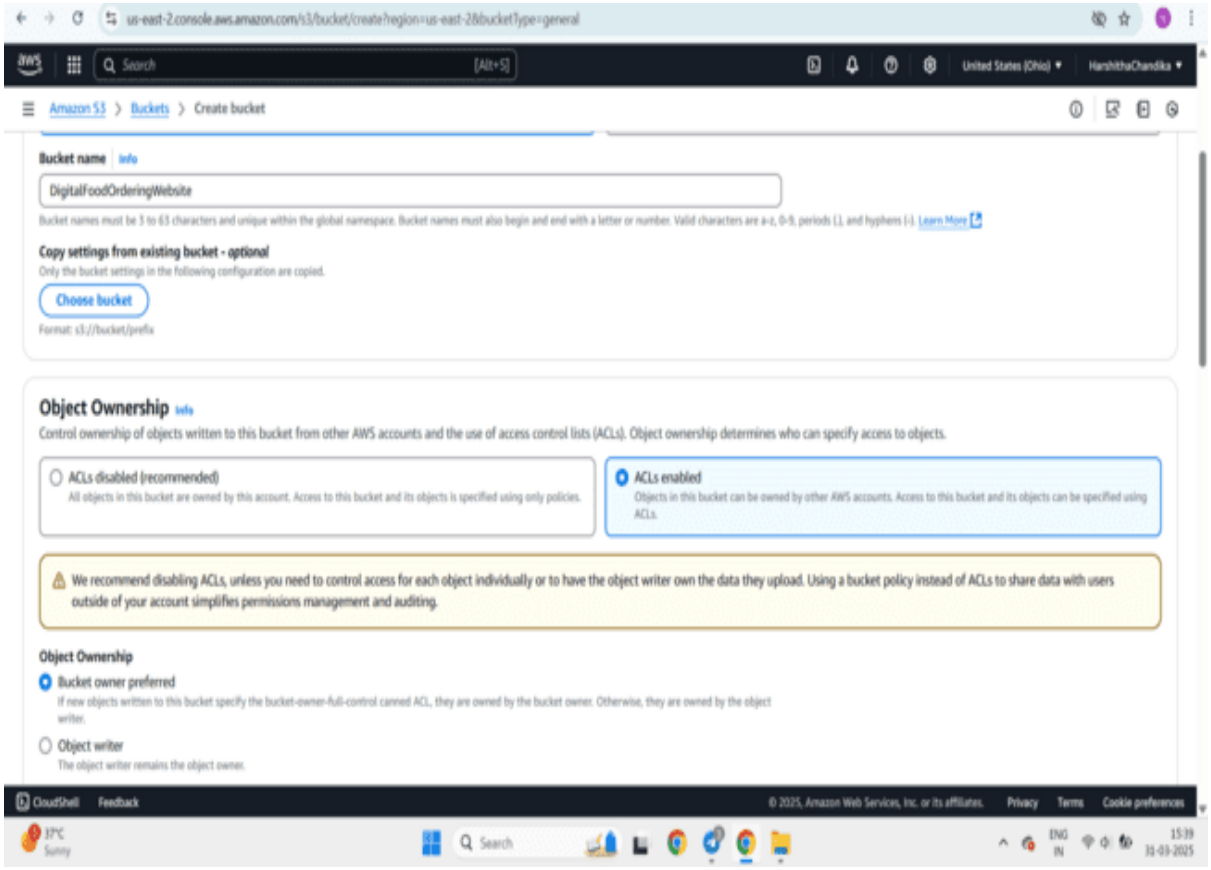


Fig1: Navigate to S3 click on Create bucket give a name for the bucket and click on ACLs Enabled to access our bucket publicly

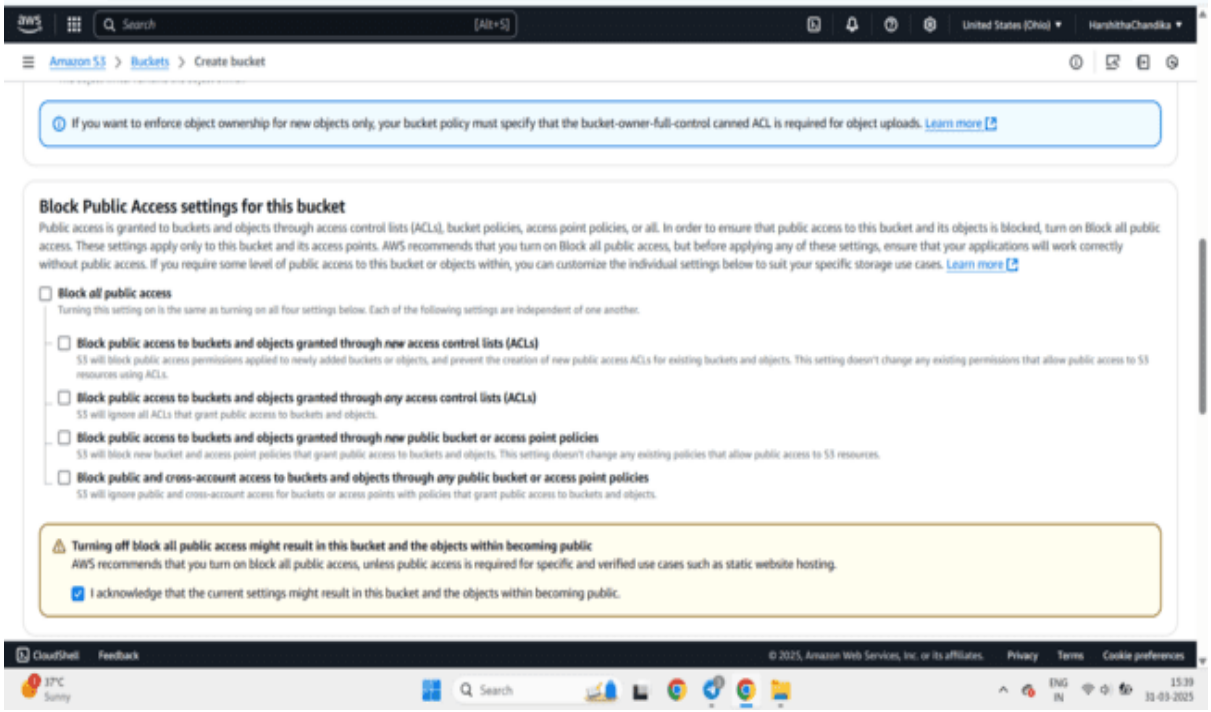


Fig2: unblock all public access and click on I acknowledge

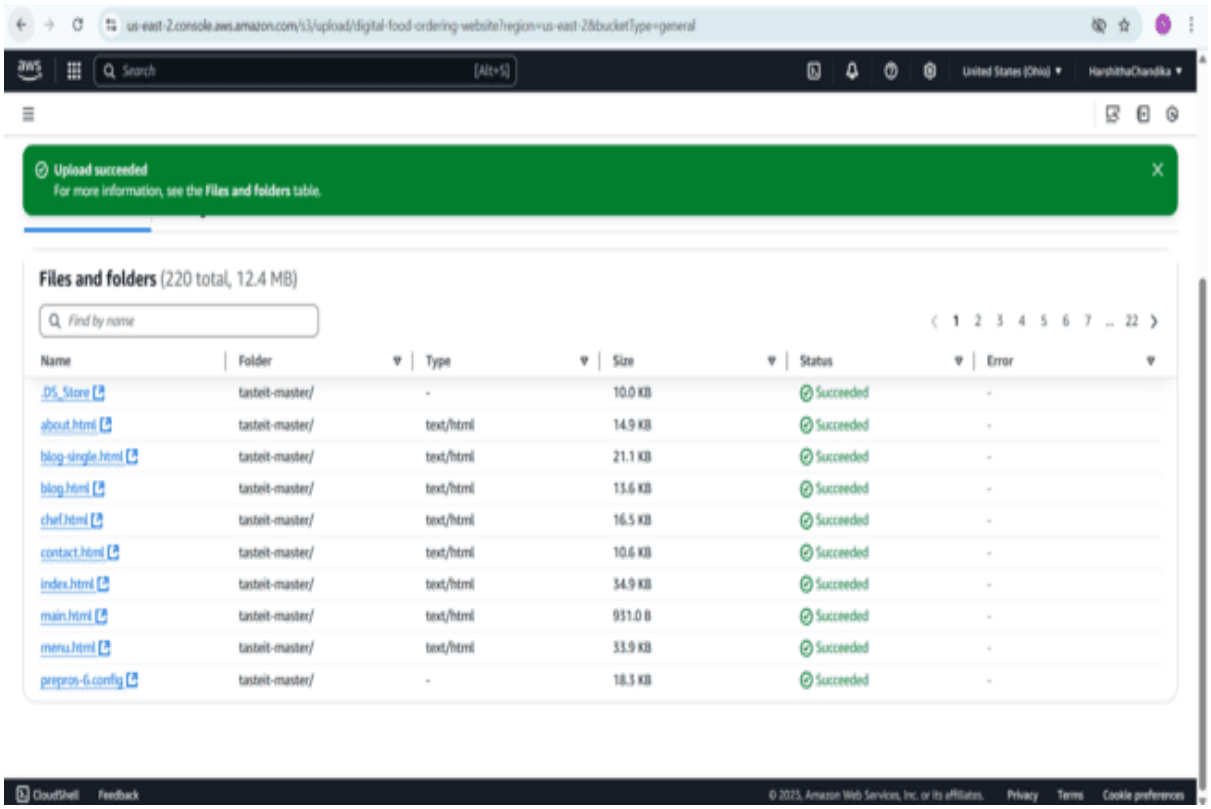


Fig3: click on bucket and upload objects select all objects go to actions click on make public using ACL

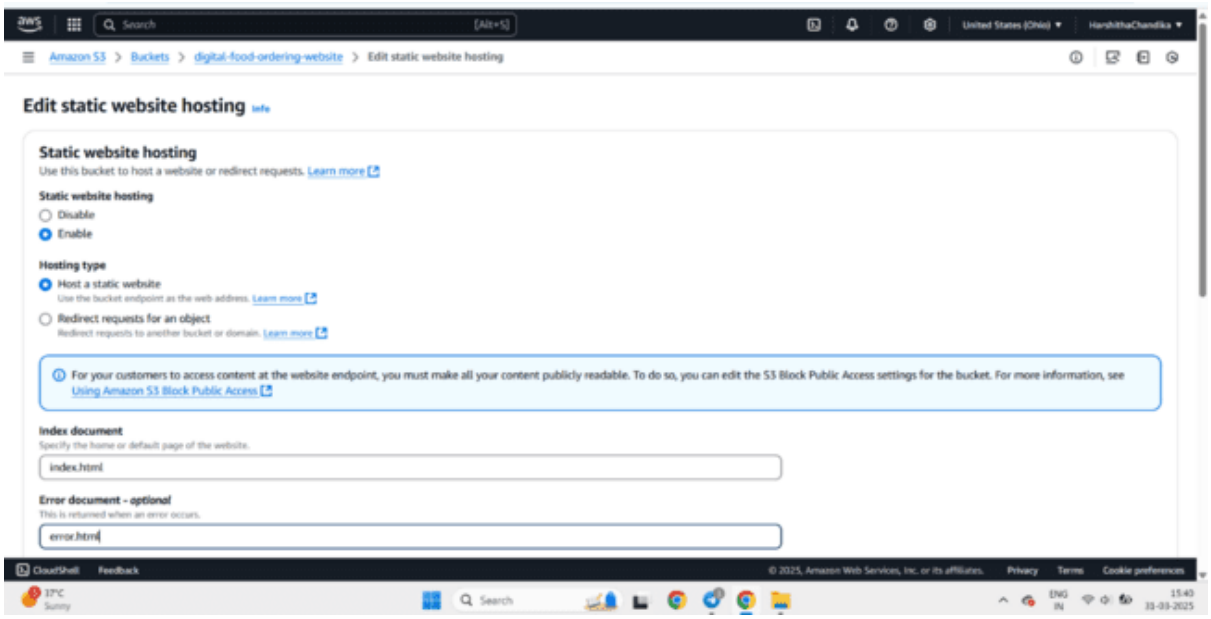


Fig4: Go to properties click on edit static website hosting click on enable give inex.html click on save changes

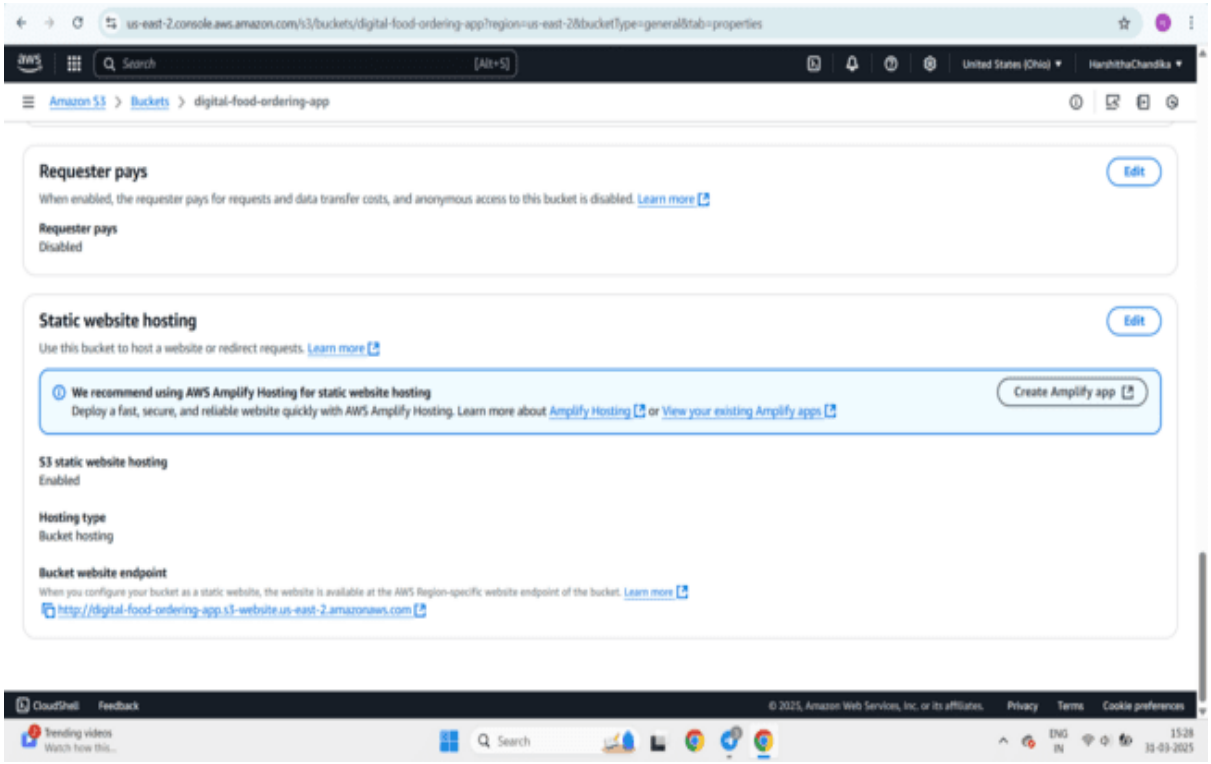


Fig5: Copy the bucket URL paste in browser and our website will open in the browser

Fig6: Home page

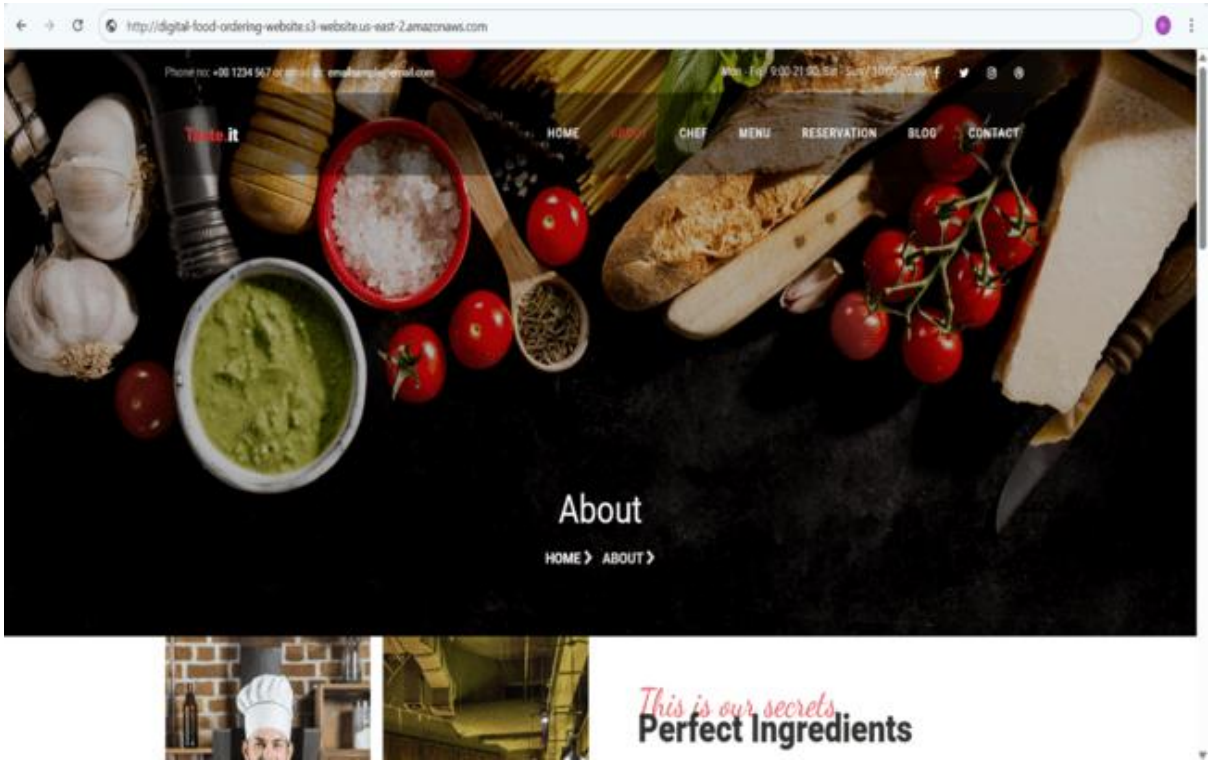


Fig7: Chefs Page

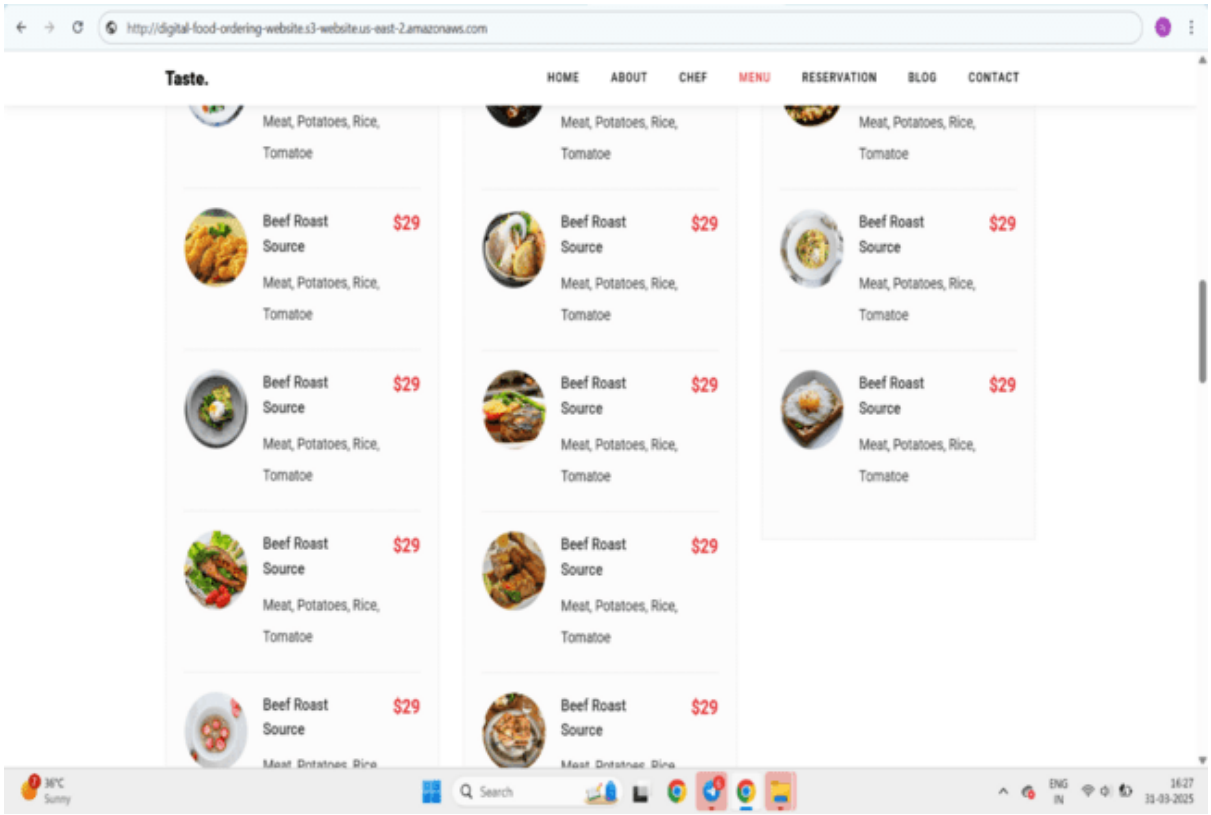


Fig 8: Menu page

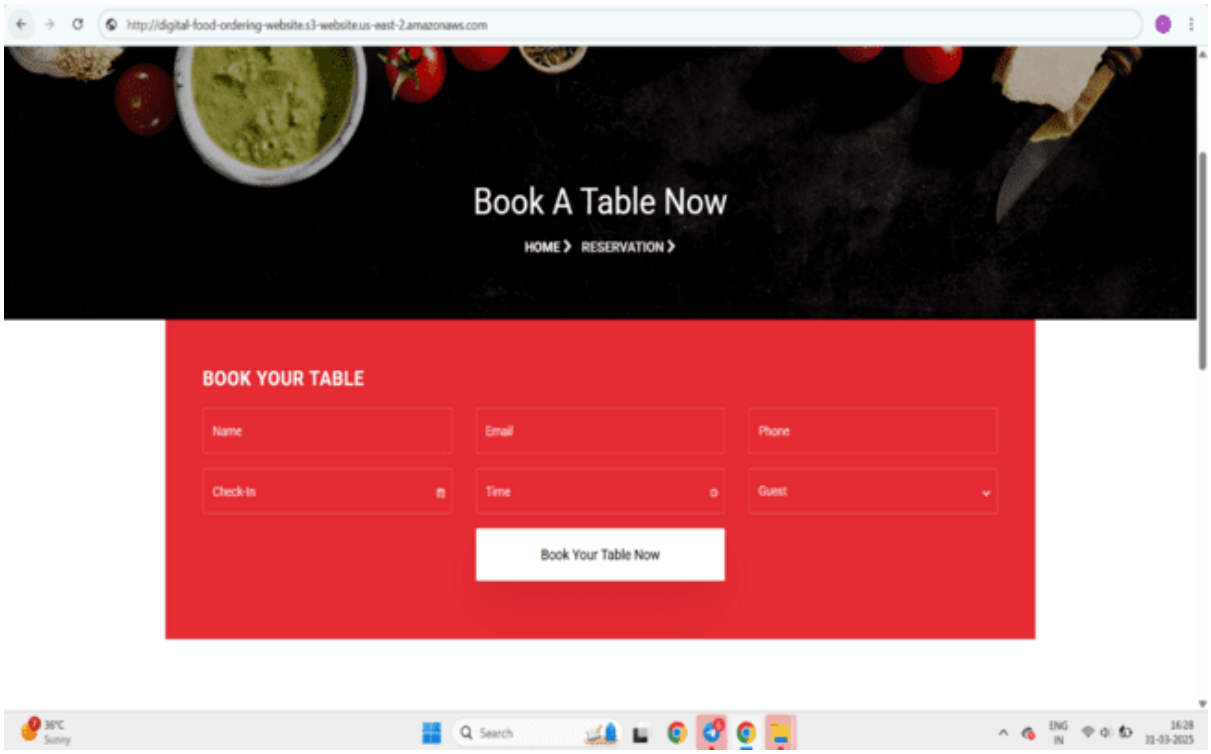


Fig 9: Booking a Table

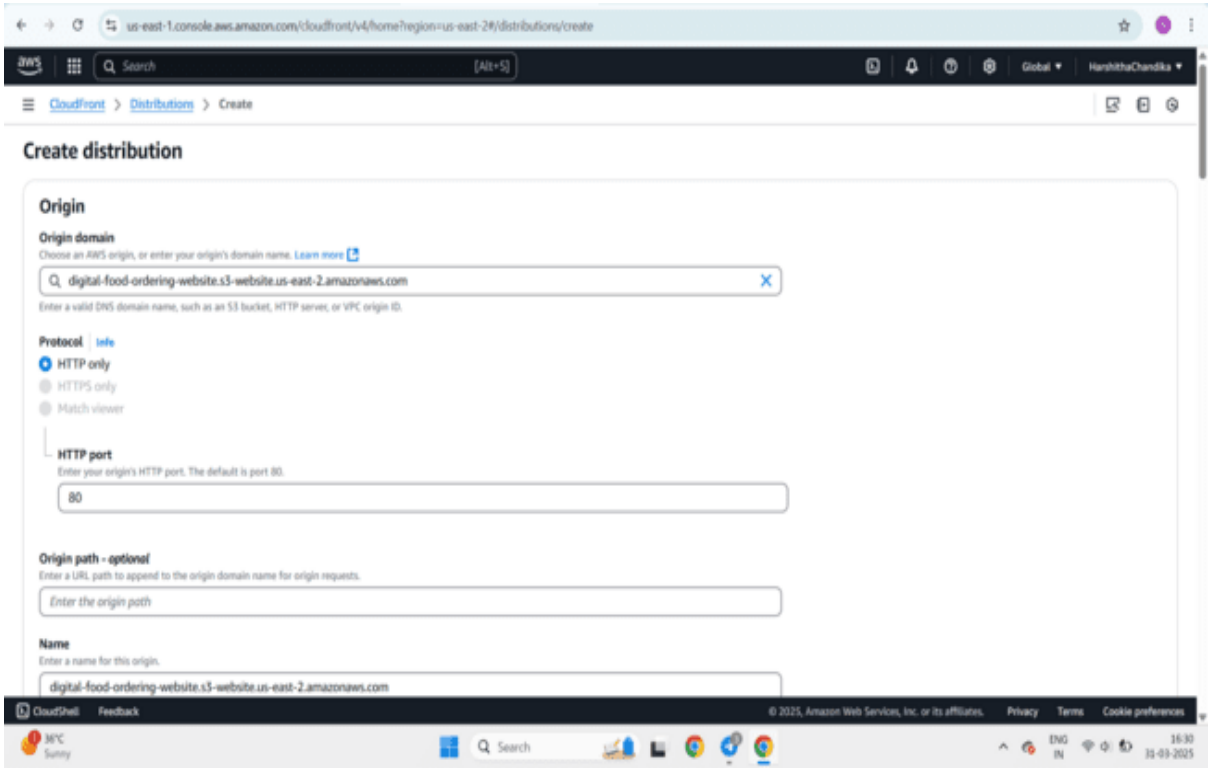


Fig 10: Navigate to CloudFront click on create distribution choose created bucket origin and select protocol http only and port 80

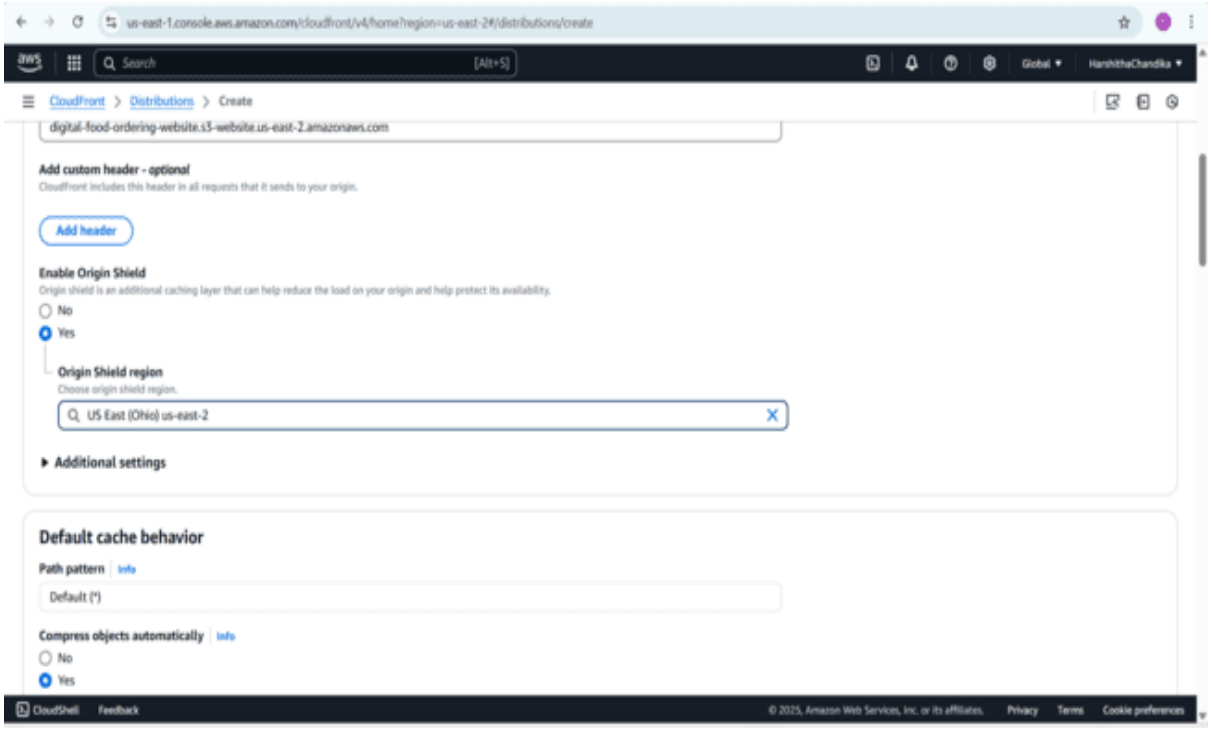


Fig 11: Enable origin shield-yes click on region select us-east-2(Ohio)region

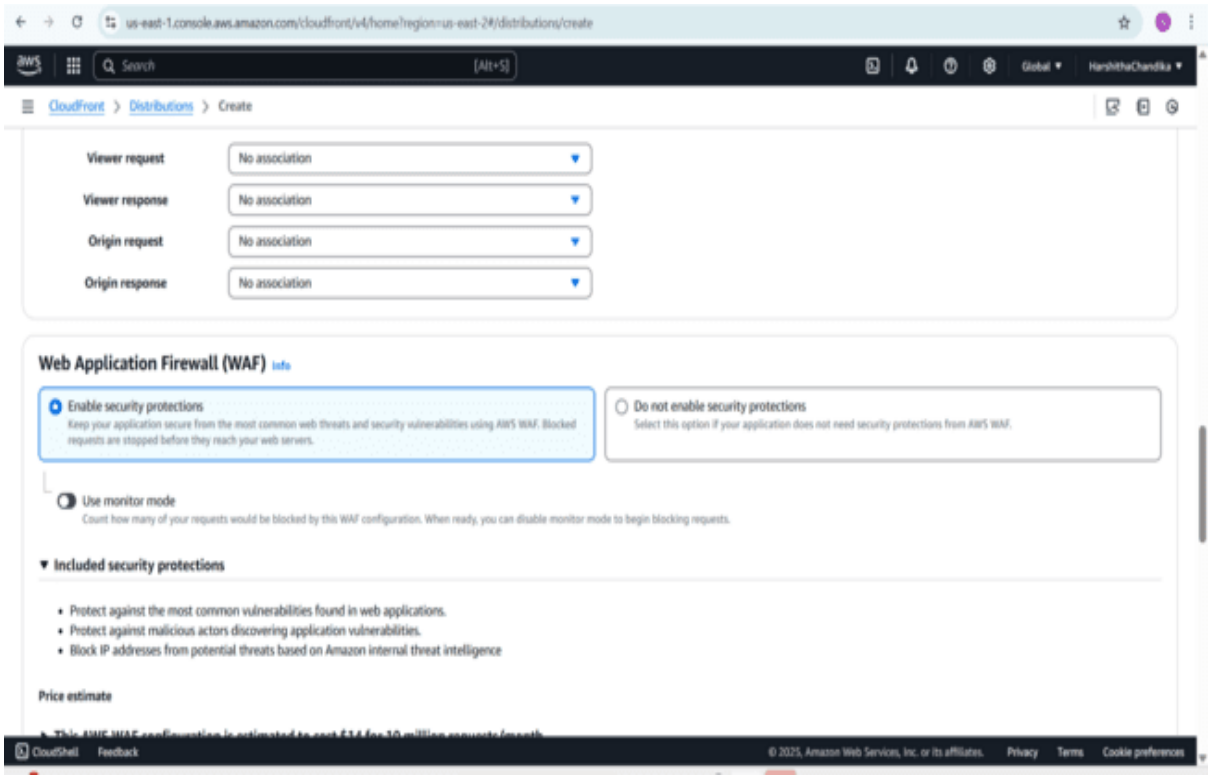


Fig 12: Enable WAF because to secure our website from threats

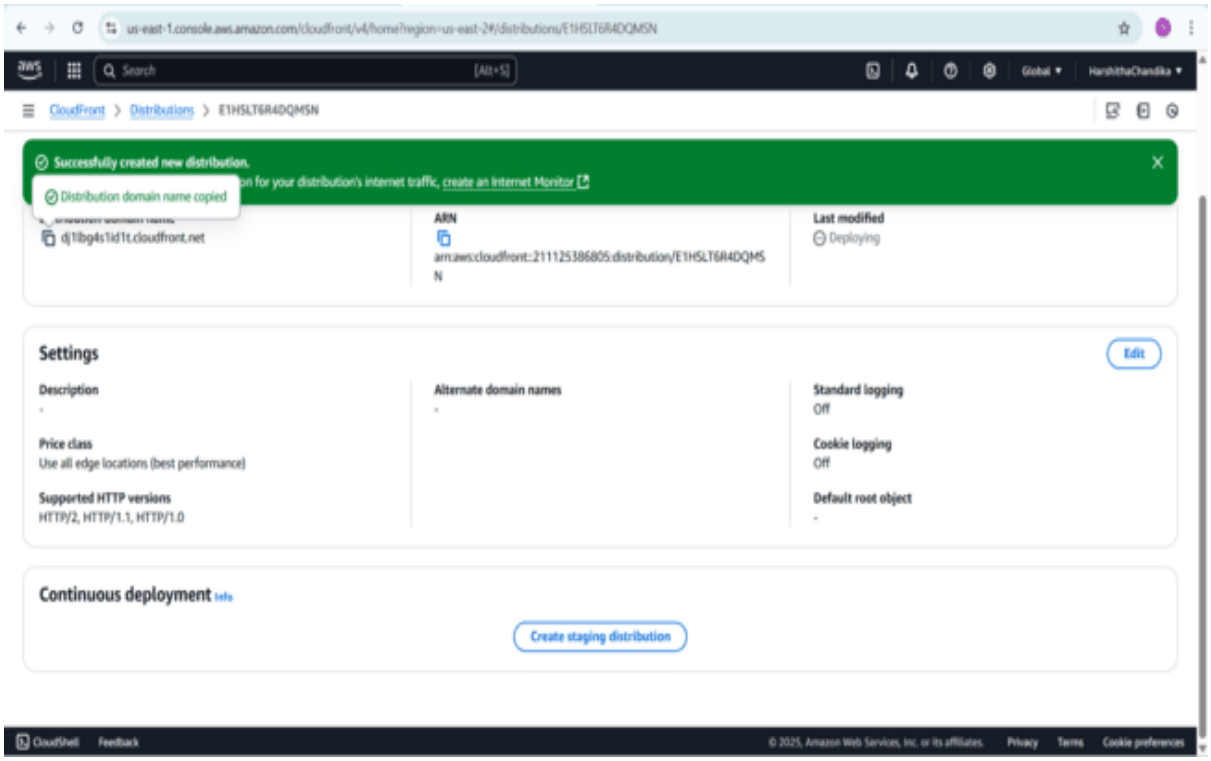


Fig 13: click on create distribution now copy the URL of the distribution

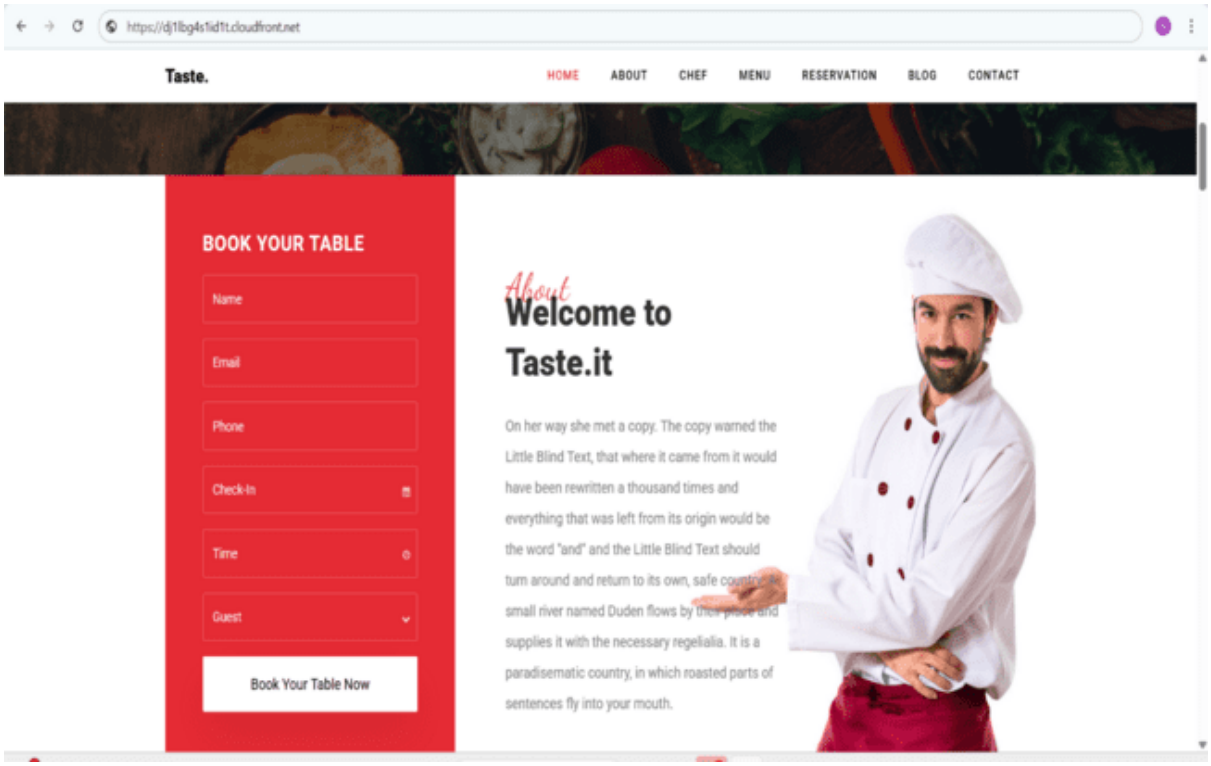


Fig 14: Paste in browser our website will open using distribution URL

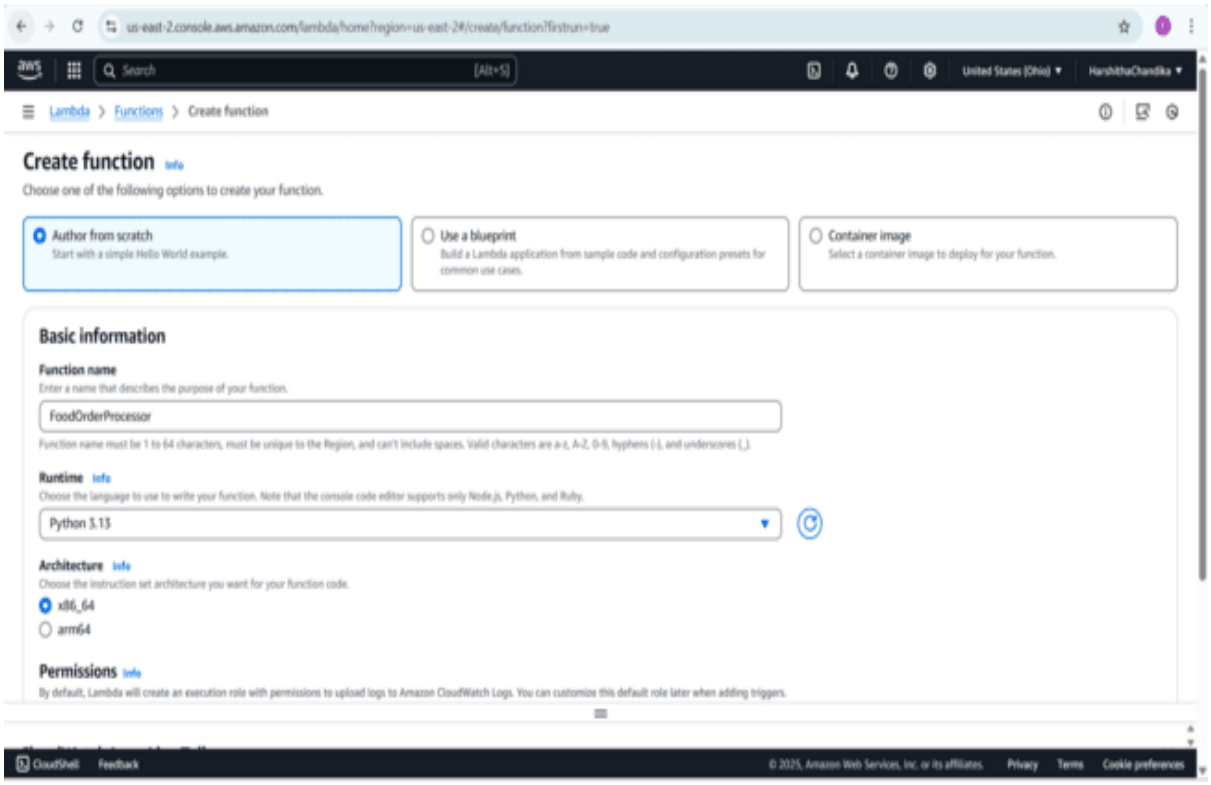


Fig 15: Navigate to lambda click on create function select author from scratch give function name. Select runtime as python

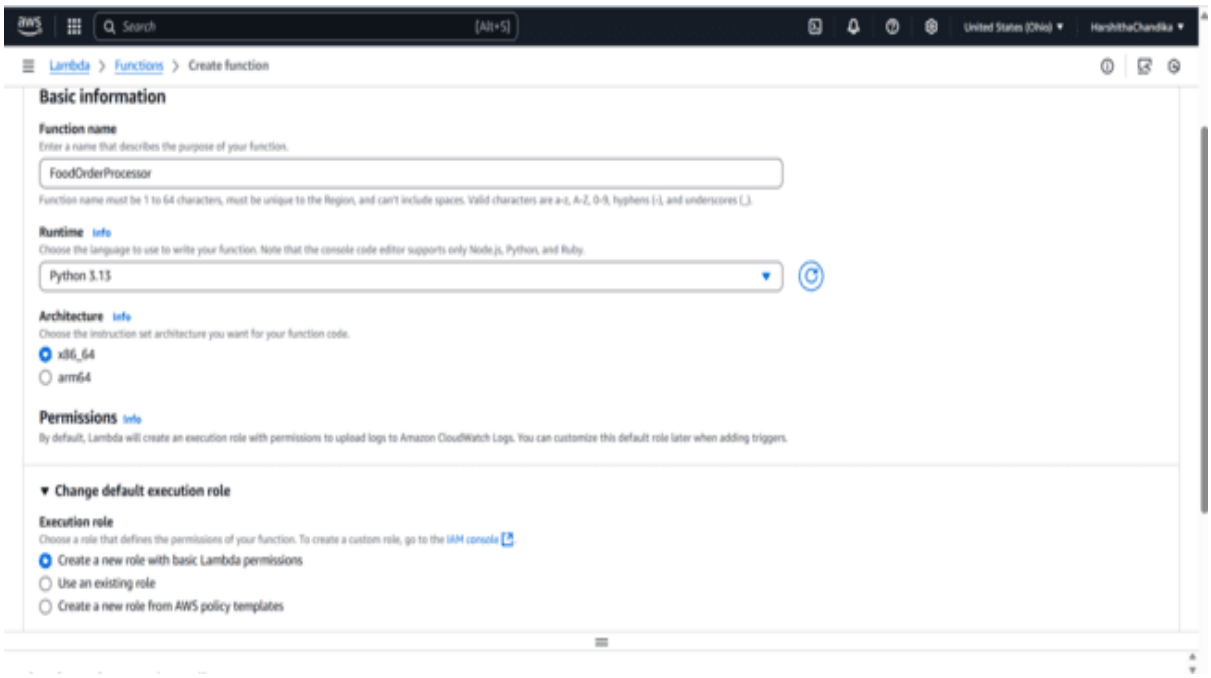


Fig 16: In permissions select create a new role with basic lambda permissions. Click on create function

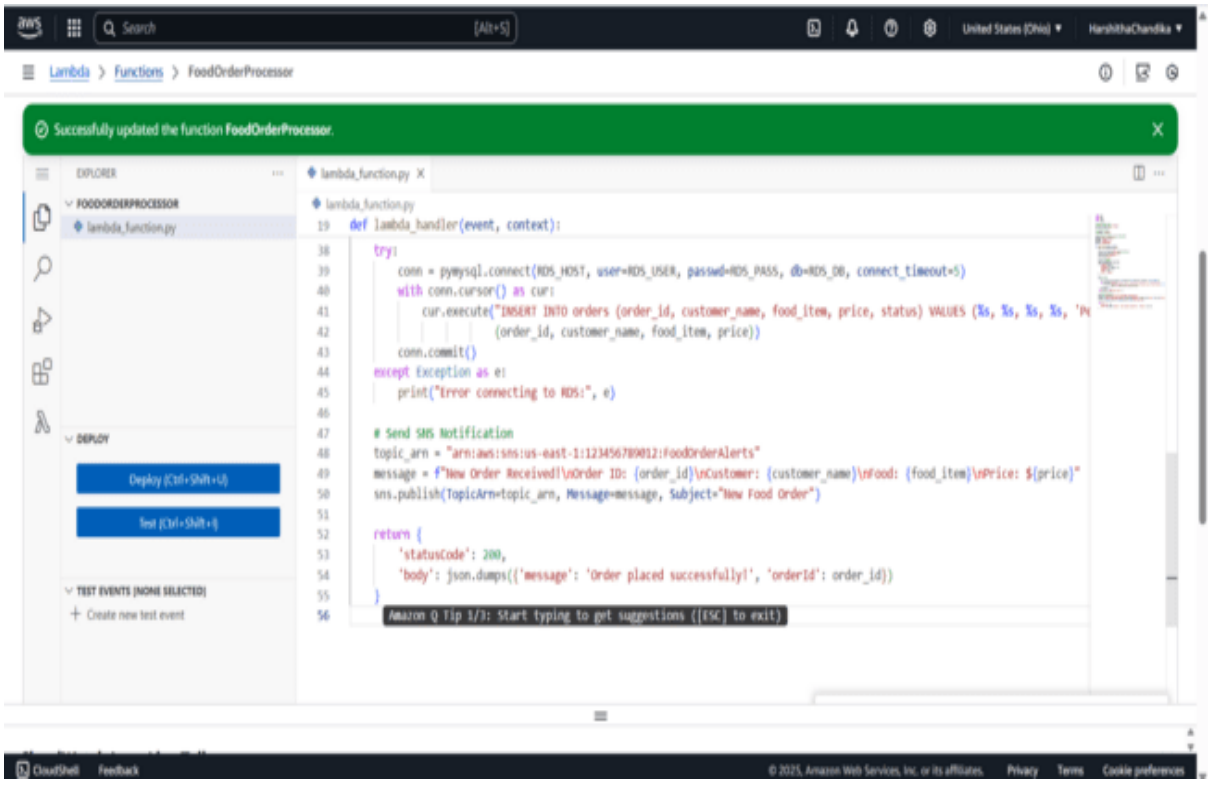


Fig 17: Give python code and click on deploy the code will successfully updated

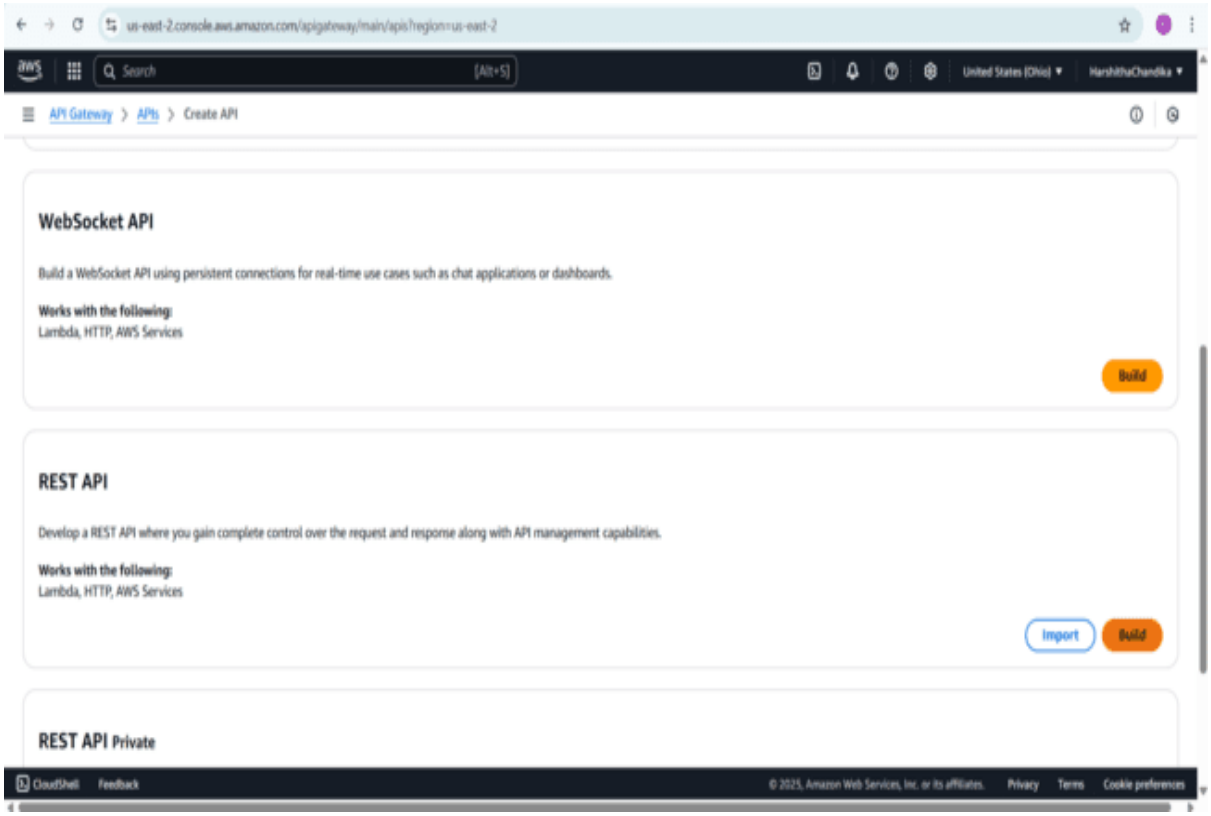


Fig 18: Navigate to API Gateway click on REST API Build

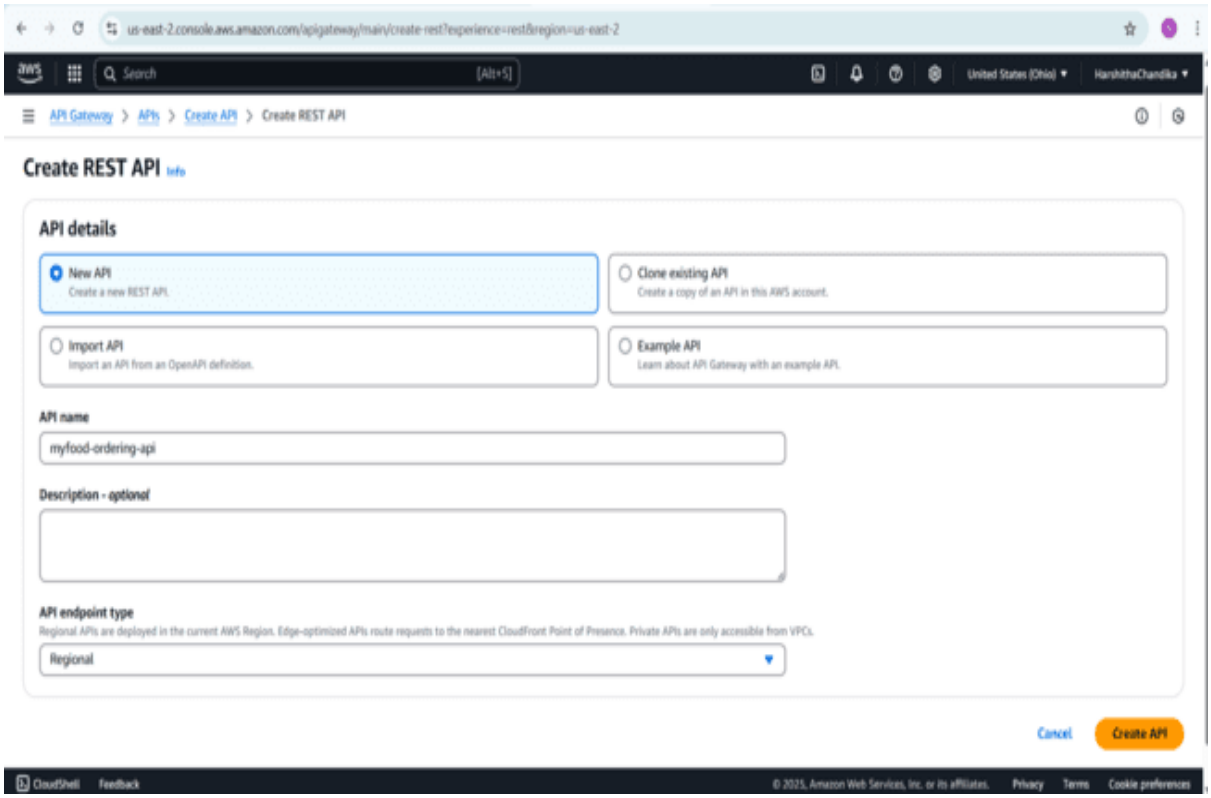


Fig 19: Give API name as myfood-ordering-api click on create API

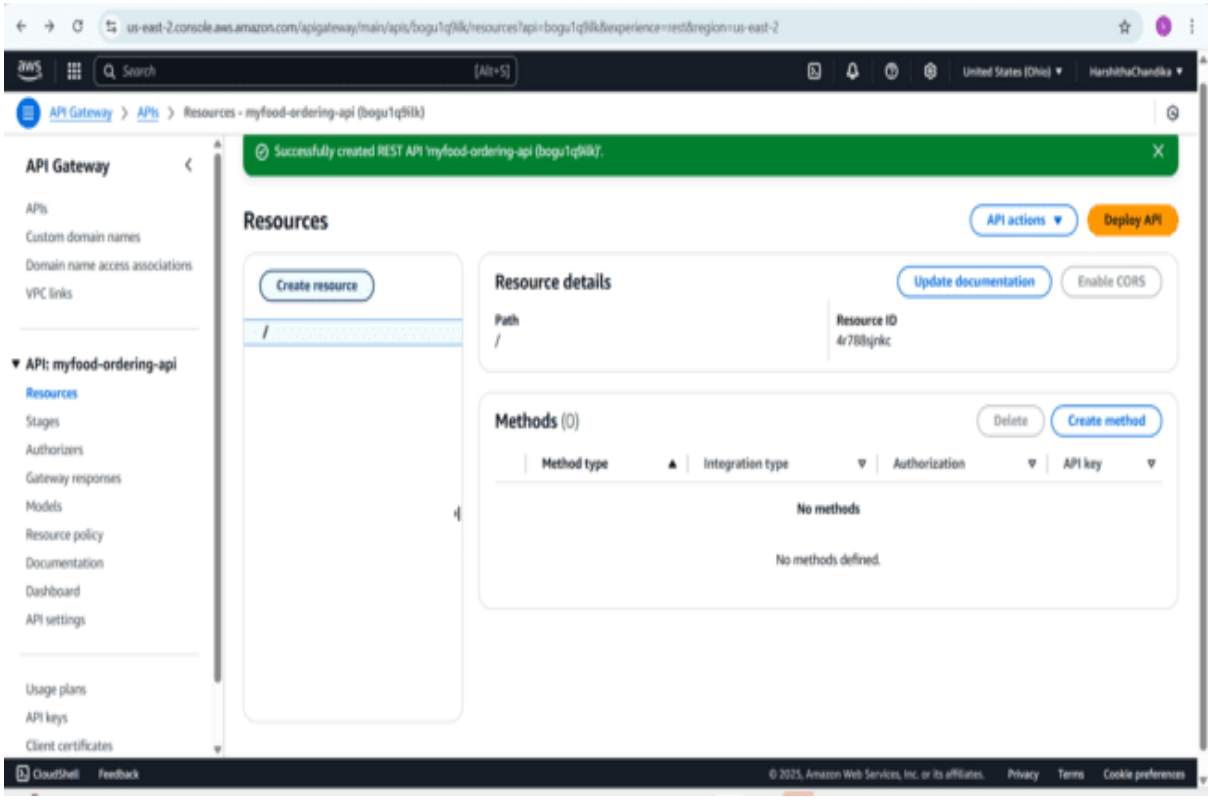


Fig 20: Go to Resources click on create Resource

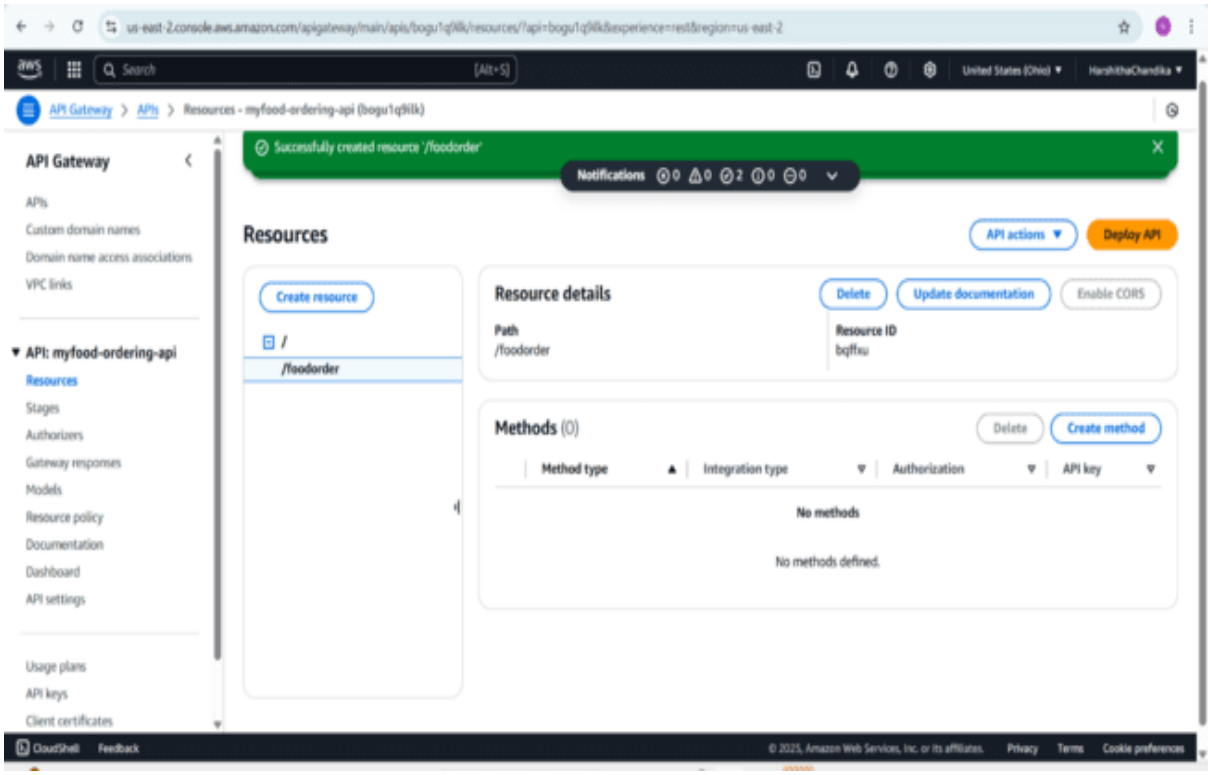


Fig 21: Give resource name and click on create resource

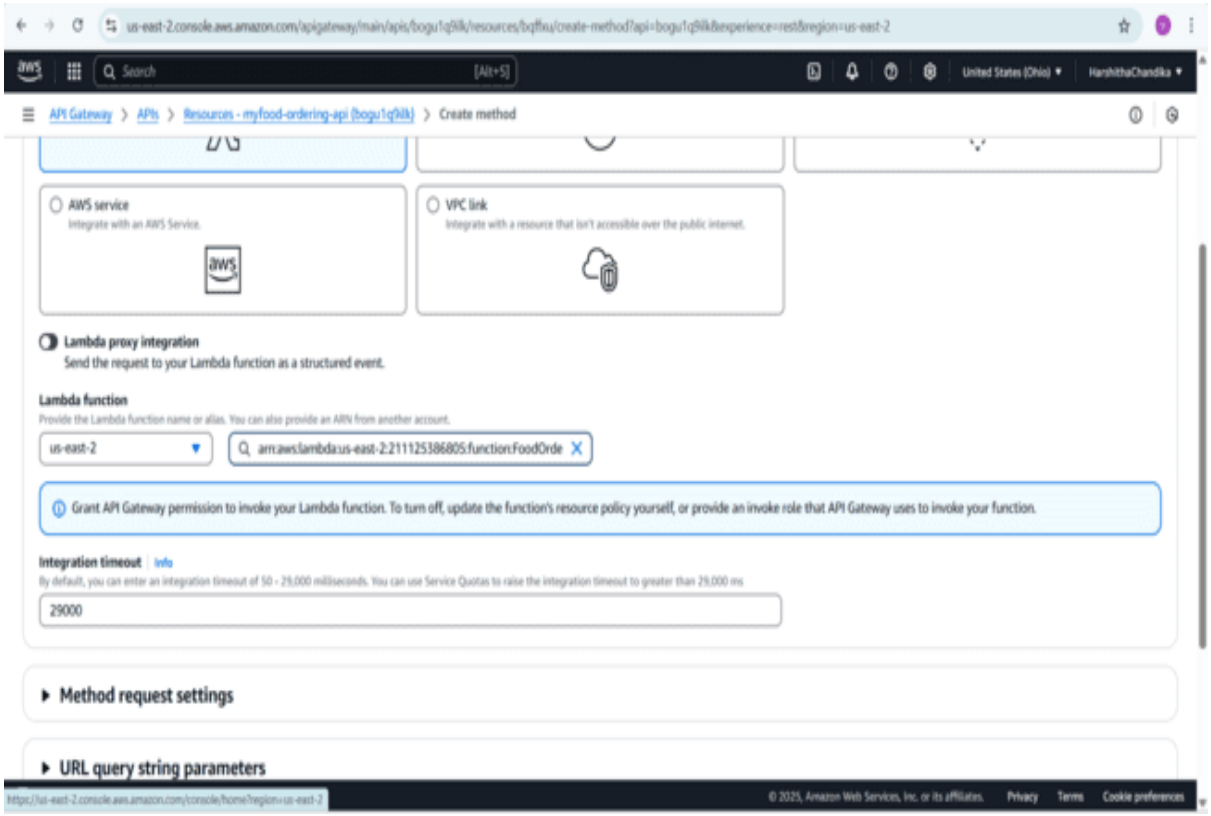


Fig 22: Go to created resource select region and lambda function

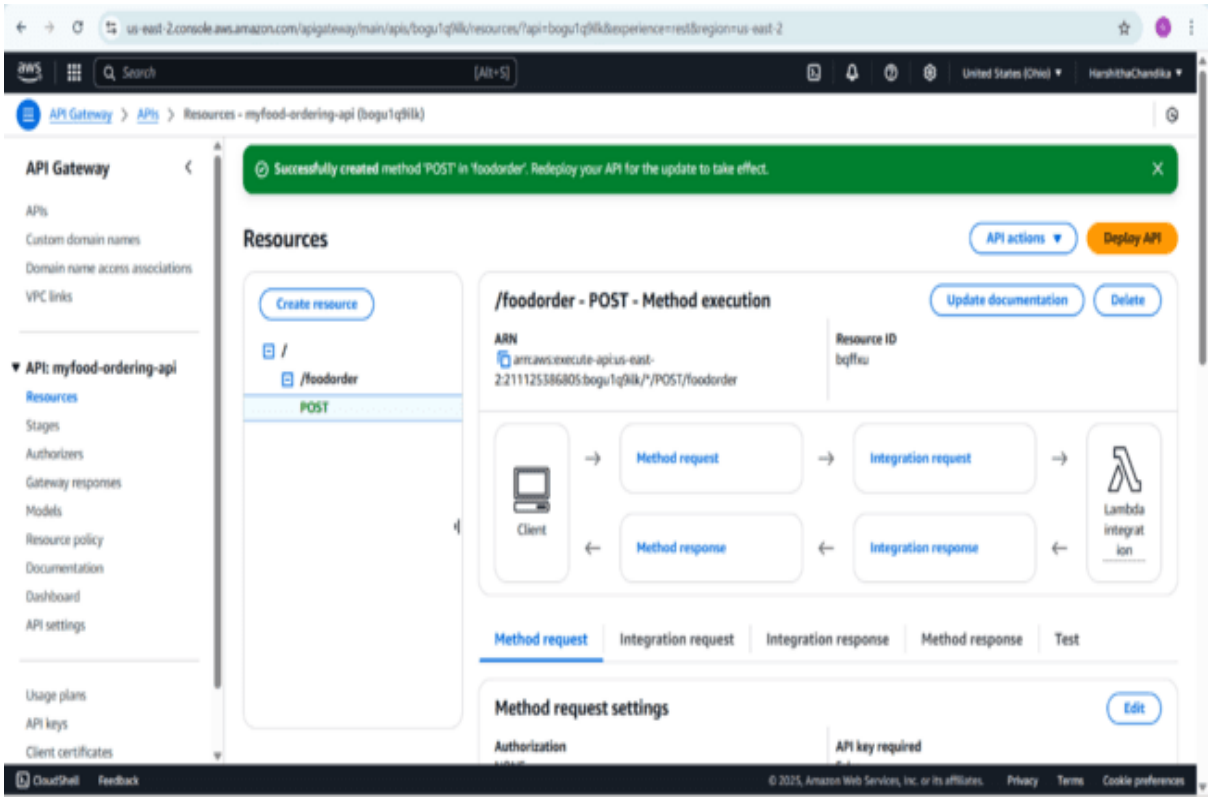


Fig 23: Under resources select POST method it successfully created under resource

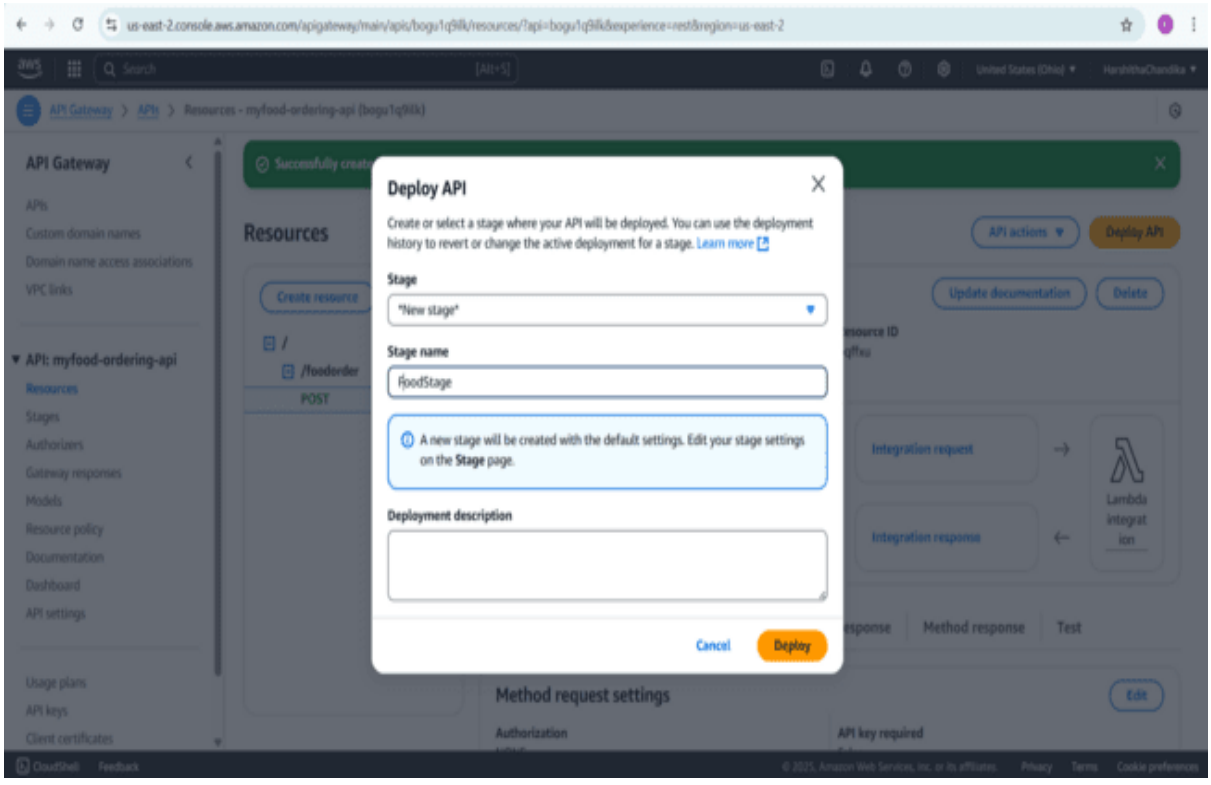


Fig 24: click on deploy API give a stage name and deploy

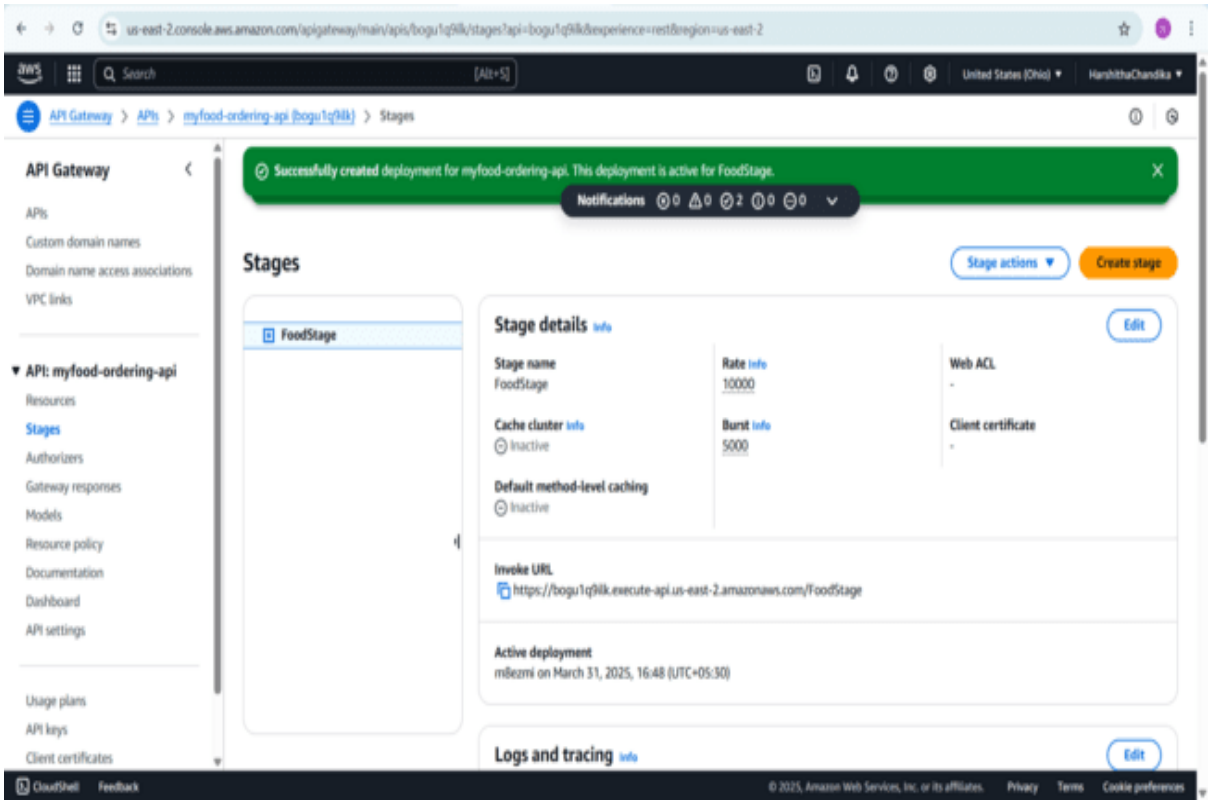


Fig 25: Now stage is successfully created copy the invoke URL paste in browser

Fig 26: when we paste in browser the message will appear in browser

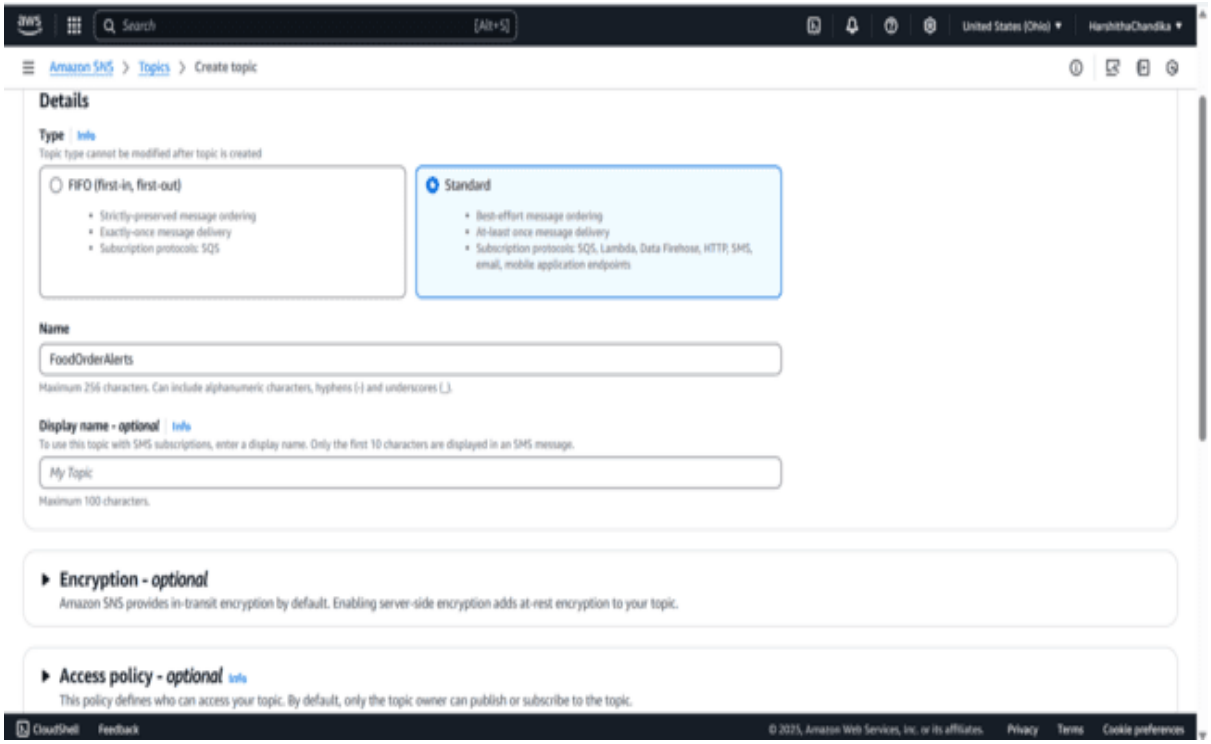


Fig 27: Navigate to SNS click create topic select standard, give topic name as FoodOrderAlerts

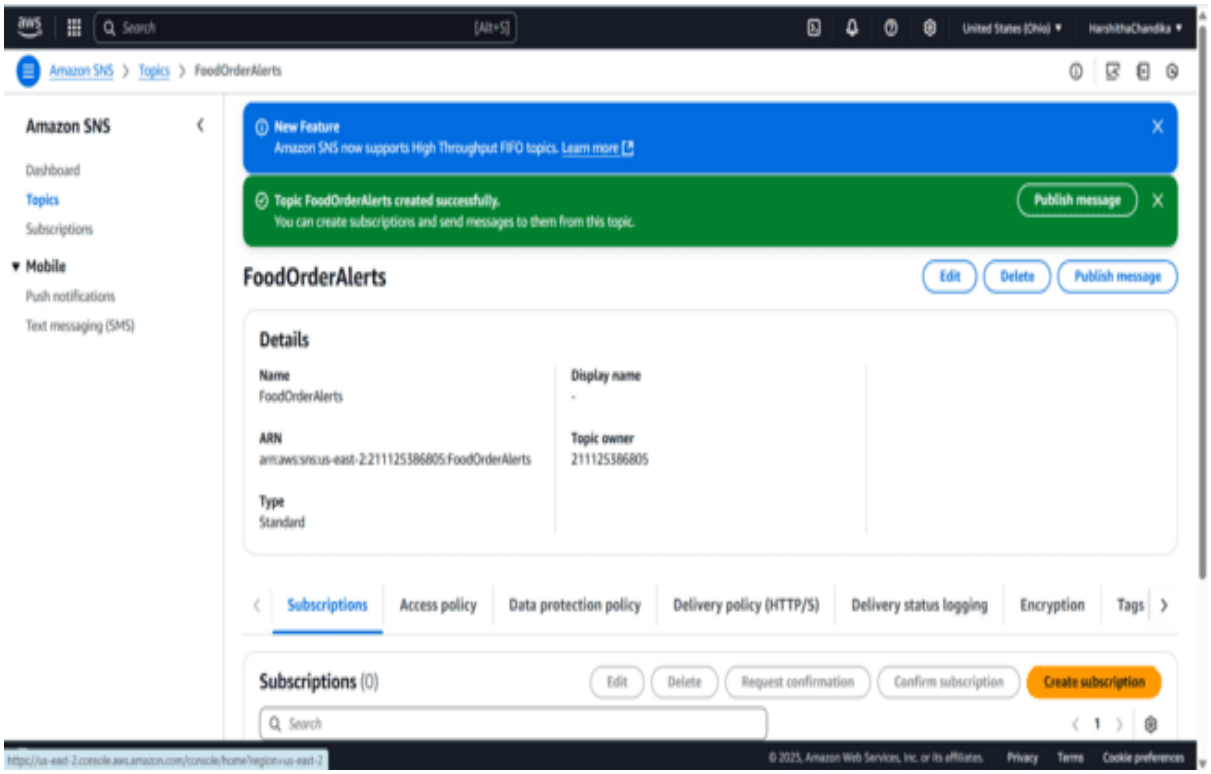


Fig 28: The topic is successfully created and click on create subscription

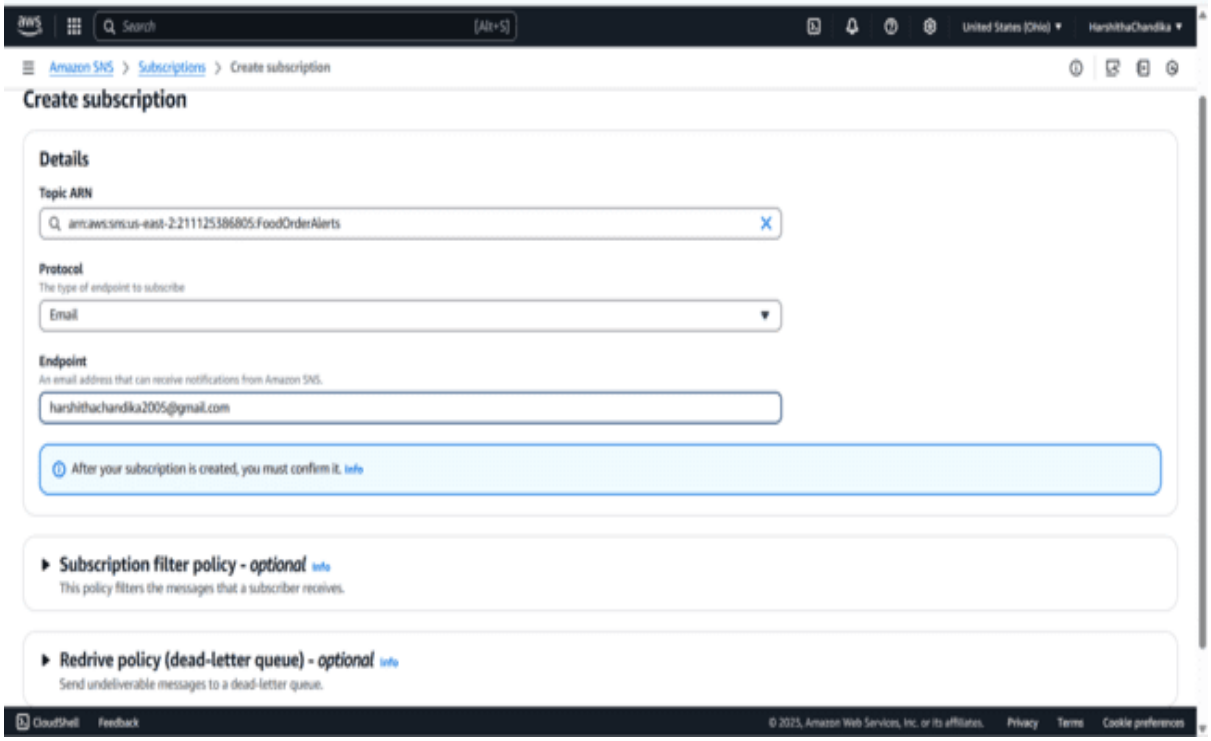


Fig 29: Give topic ARN select protocol email give endpoint as your personal email click on create

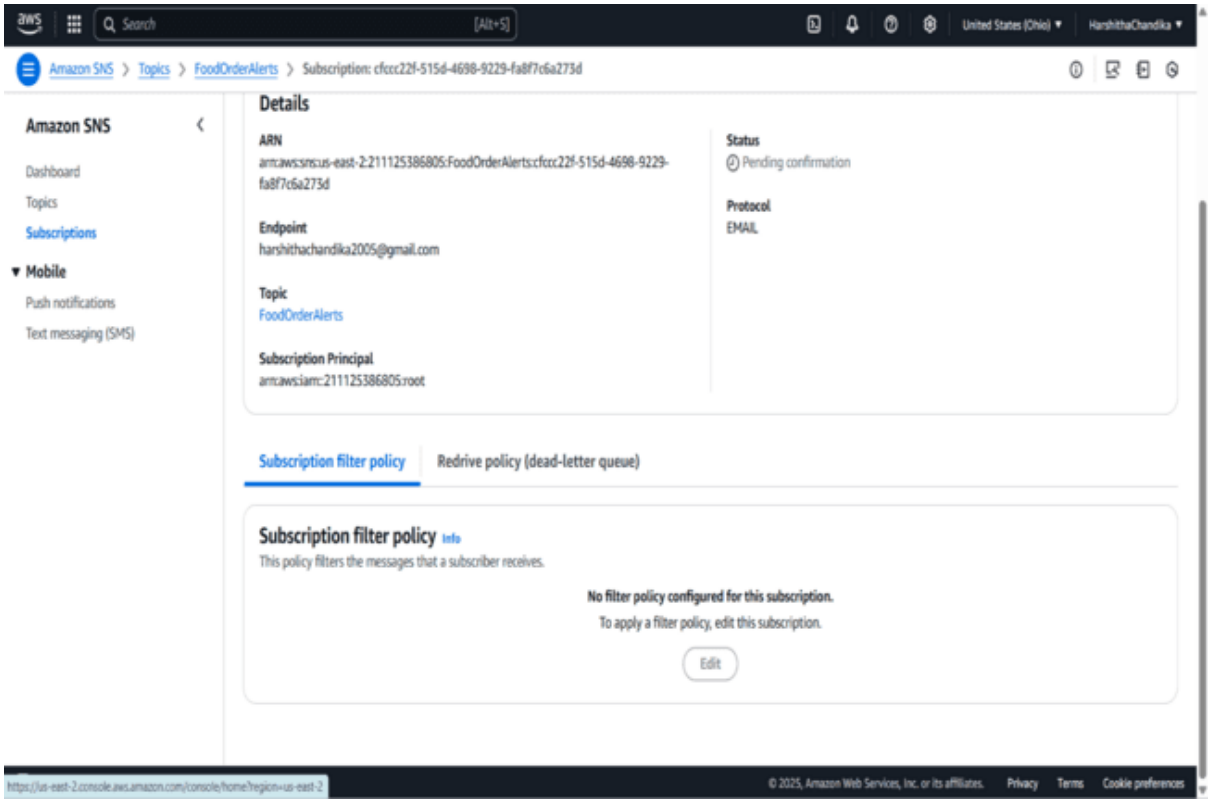


Fig 30: now subscription successfully created

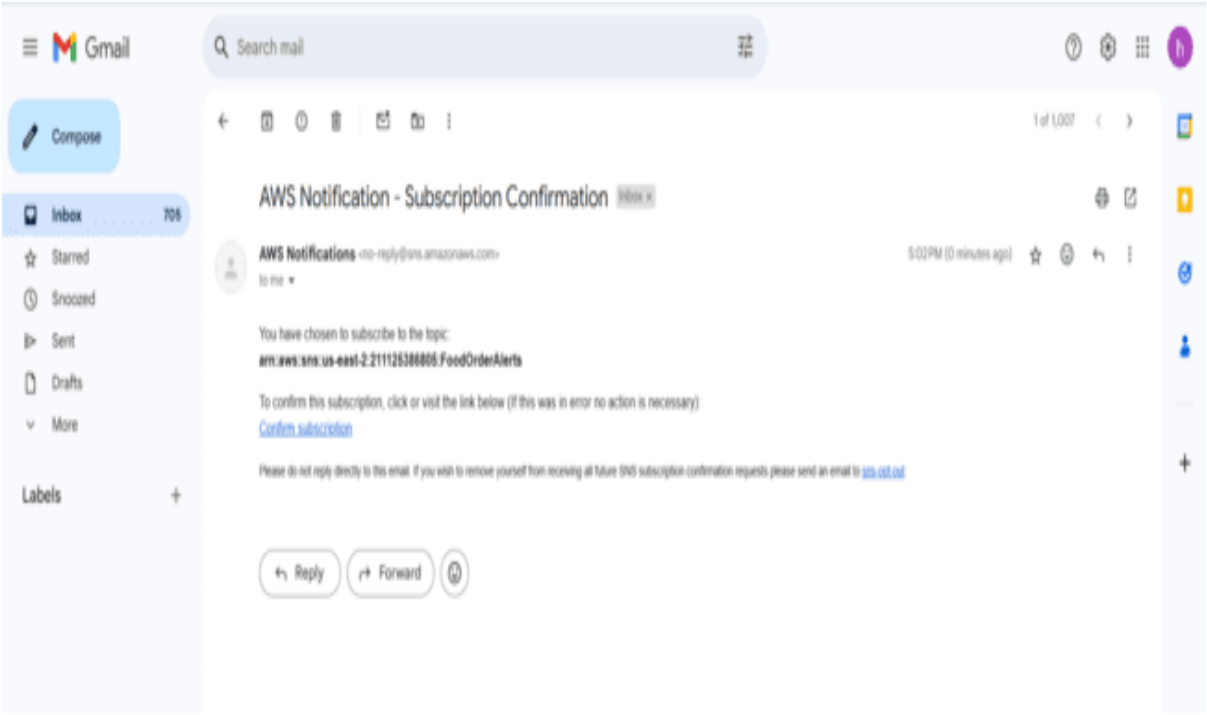


Fig 31: Go to email given in subscription click on confirm subscription

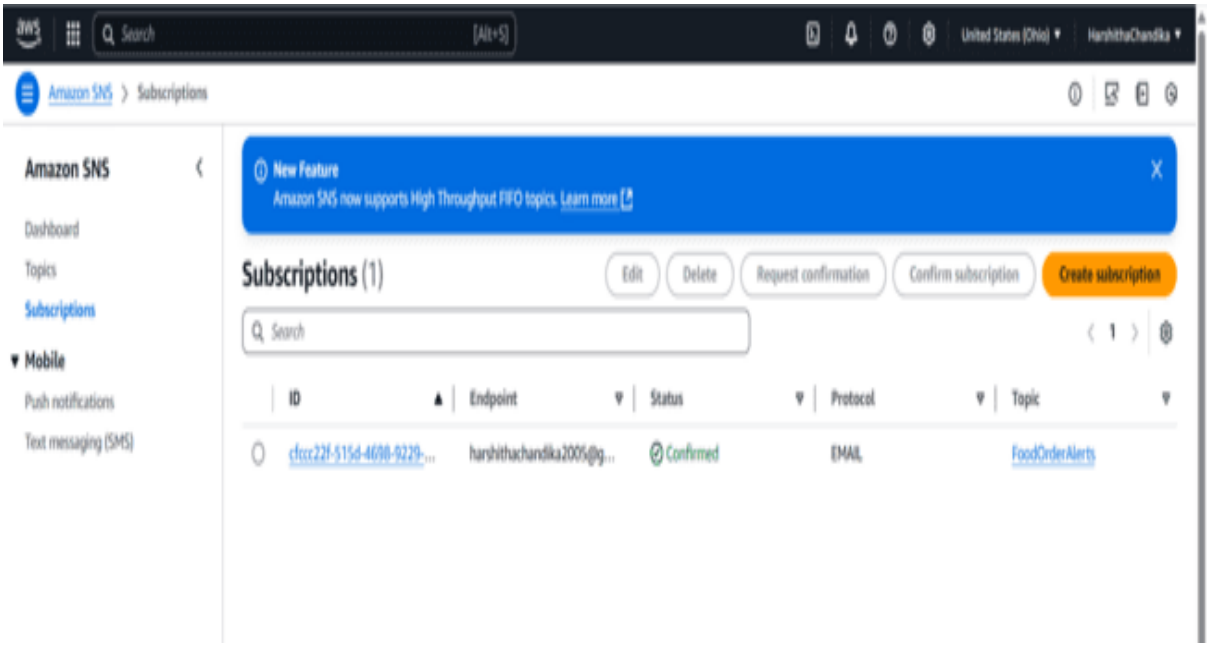


Fig 32: now the status is confirmed

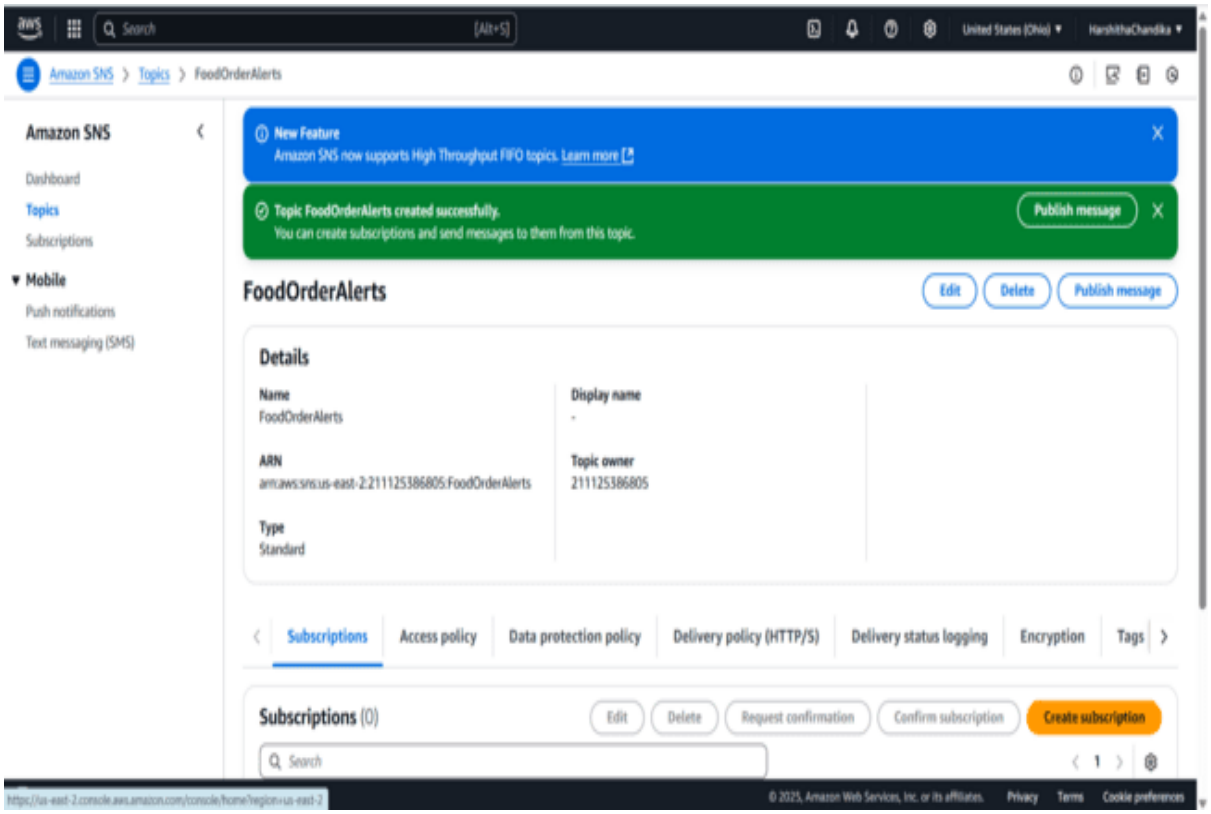
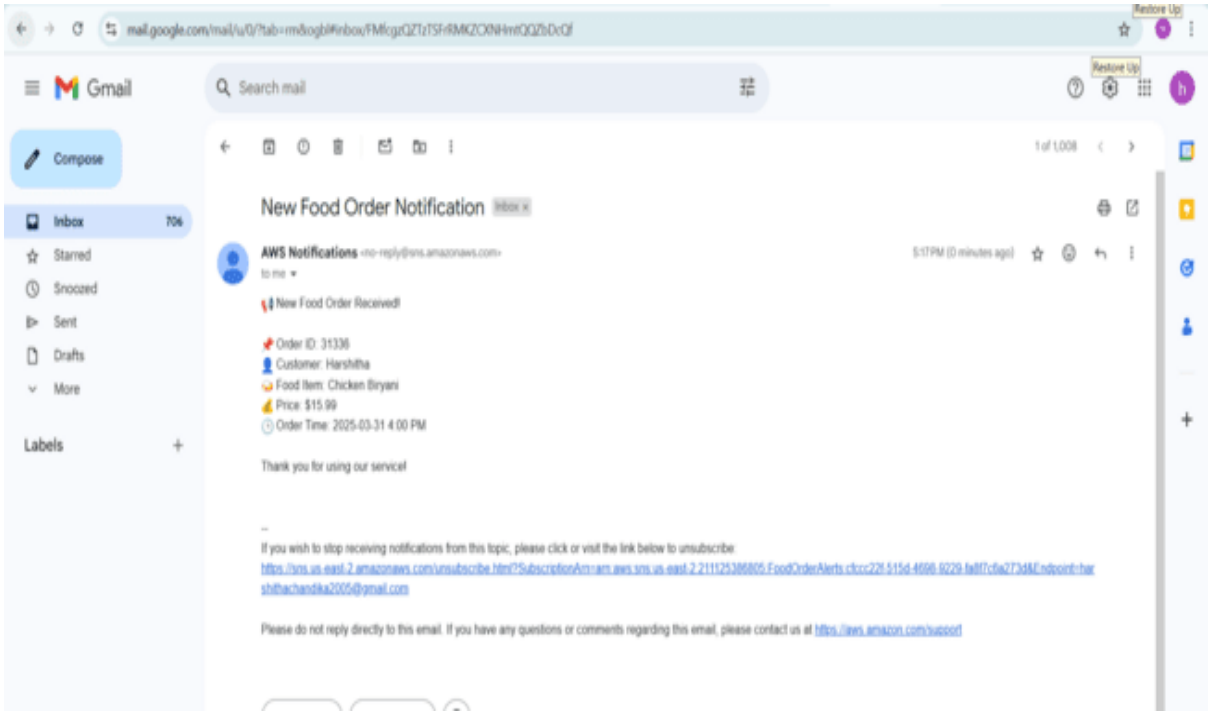


Fig 33:Go to topics click on publish message give subject as Food Order Notification and give the message in body section click on publish



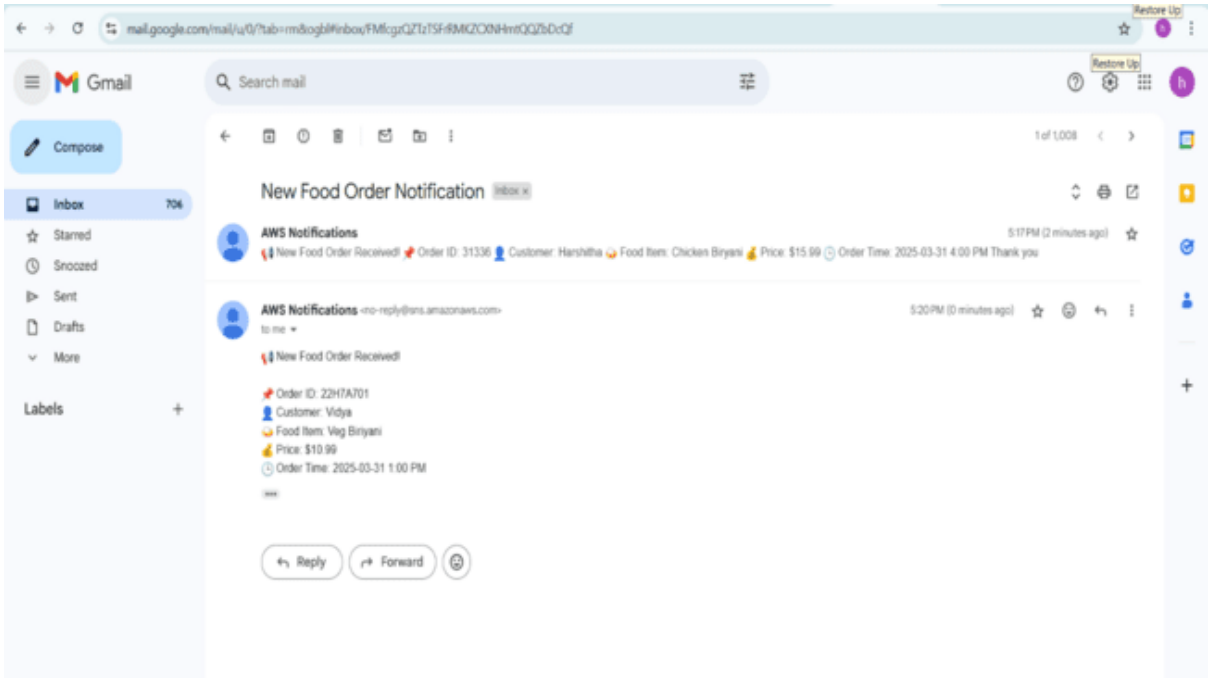


Fig 34&35: Two food order notifications I get to my mail

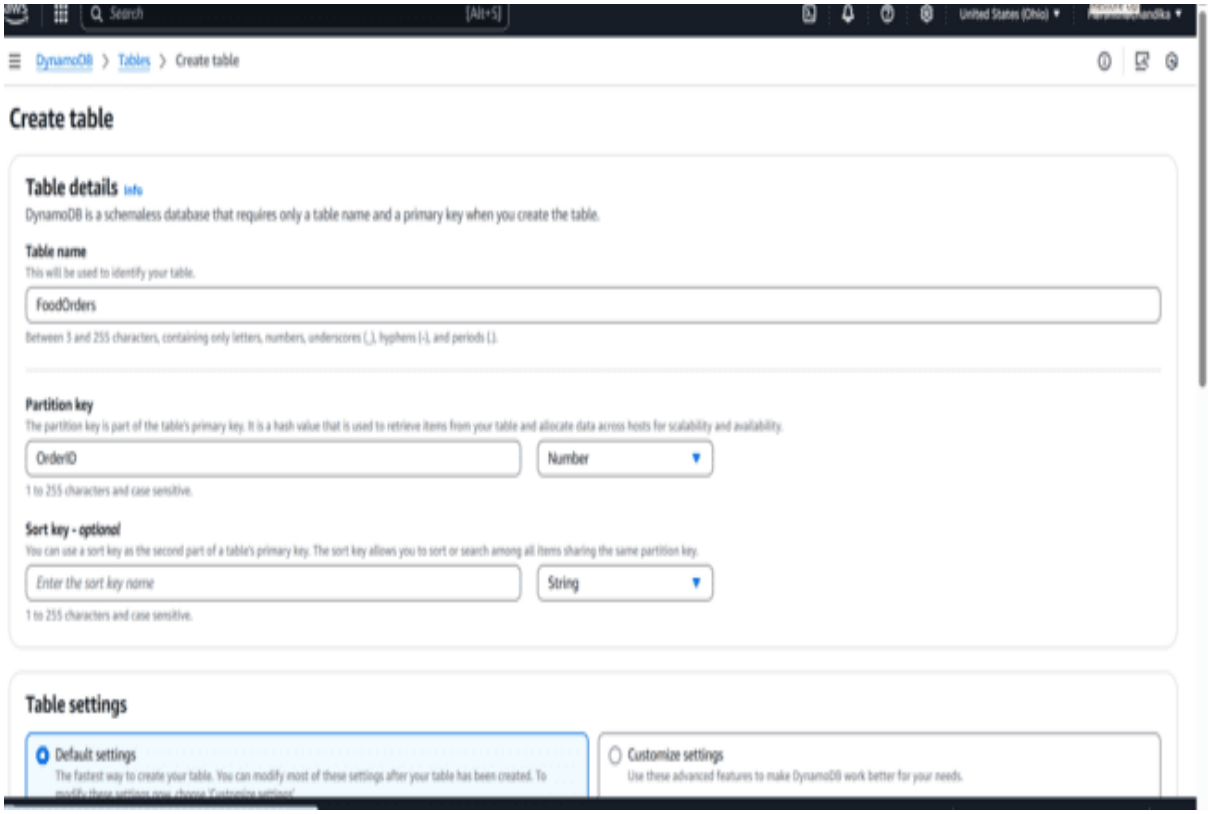


Fig 36: Navigate to DynamoDB click on create table give table name and partition key click on create table

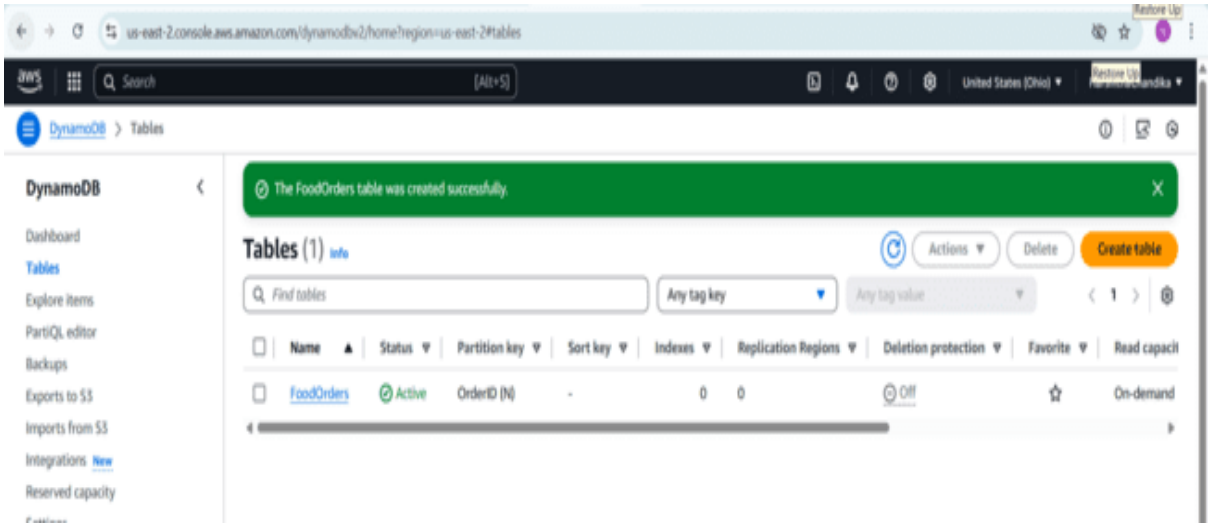


Fig 37: Now the table is created

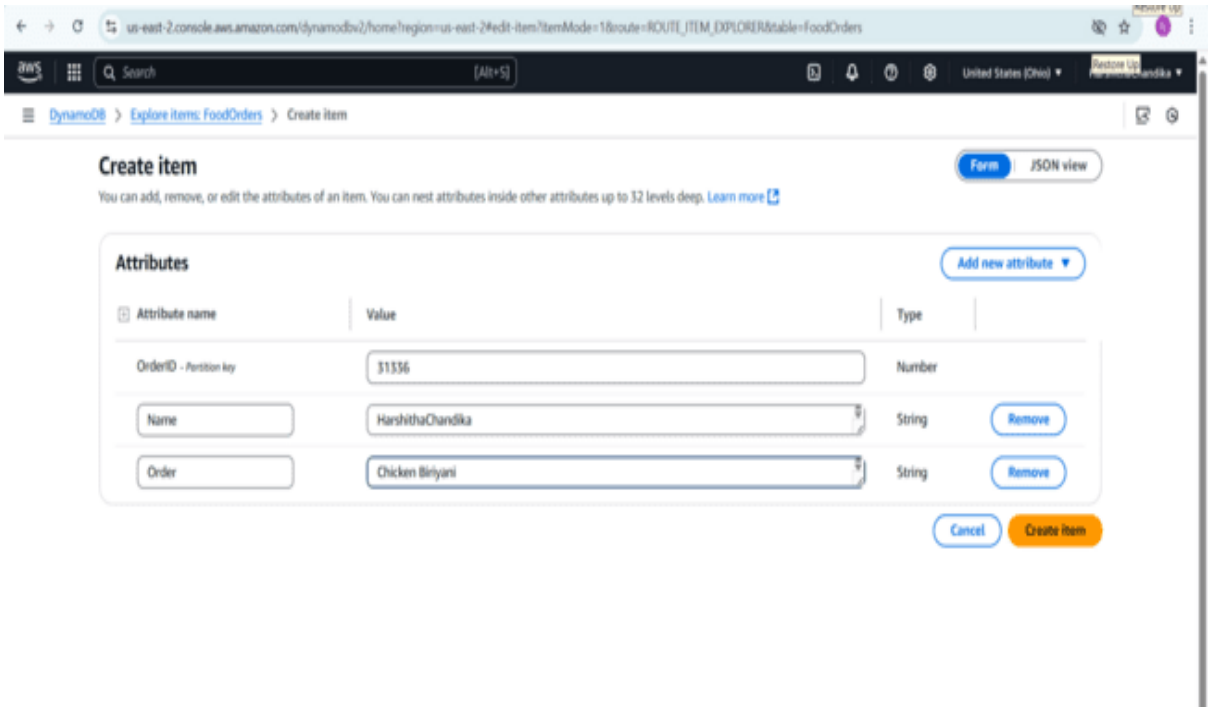


Fig 38: click on created table ,click on explore items , click on create item and add attributes

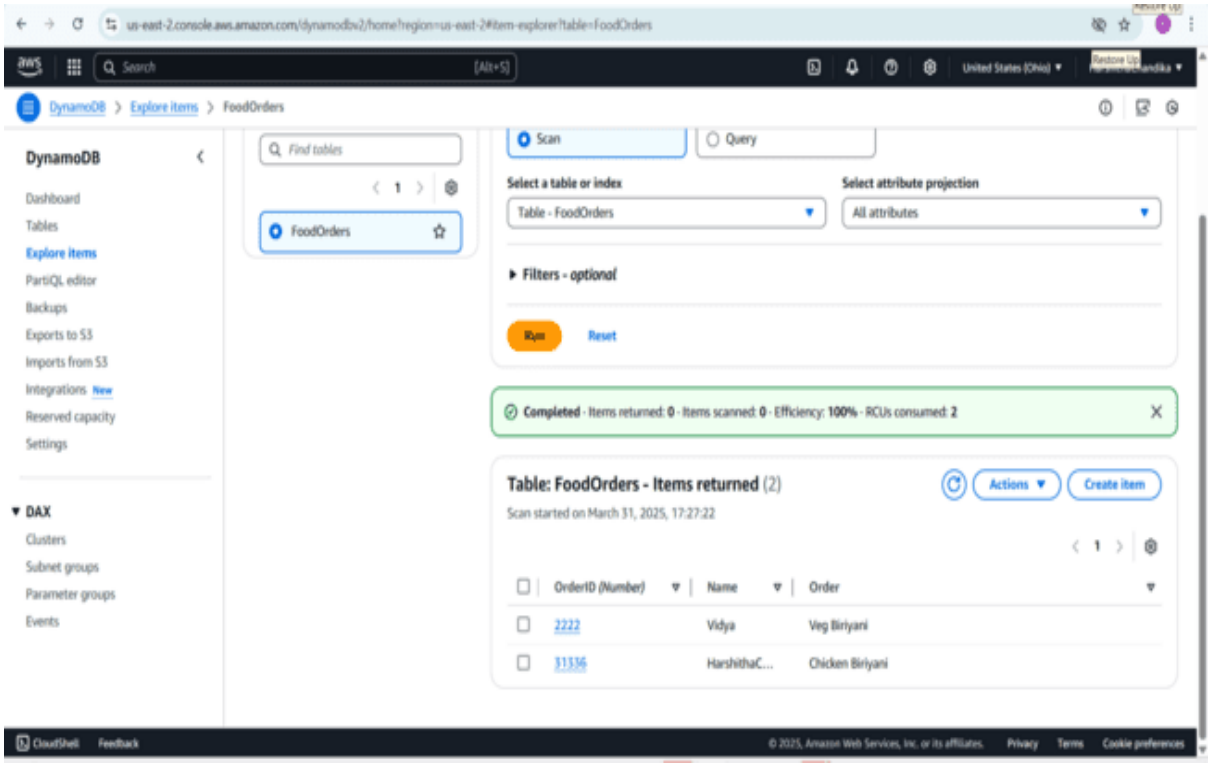


Fig 39: Now created attributes will appear here

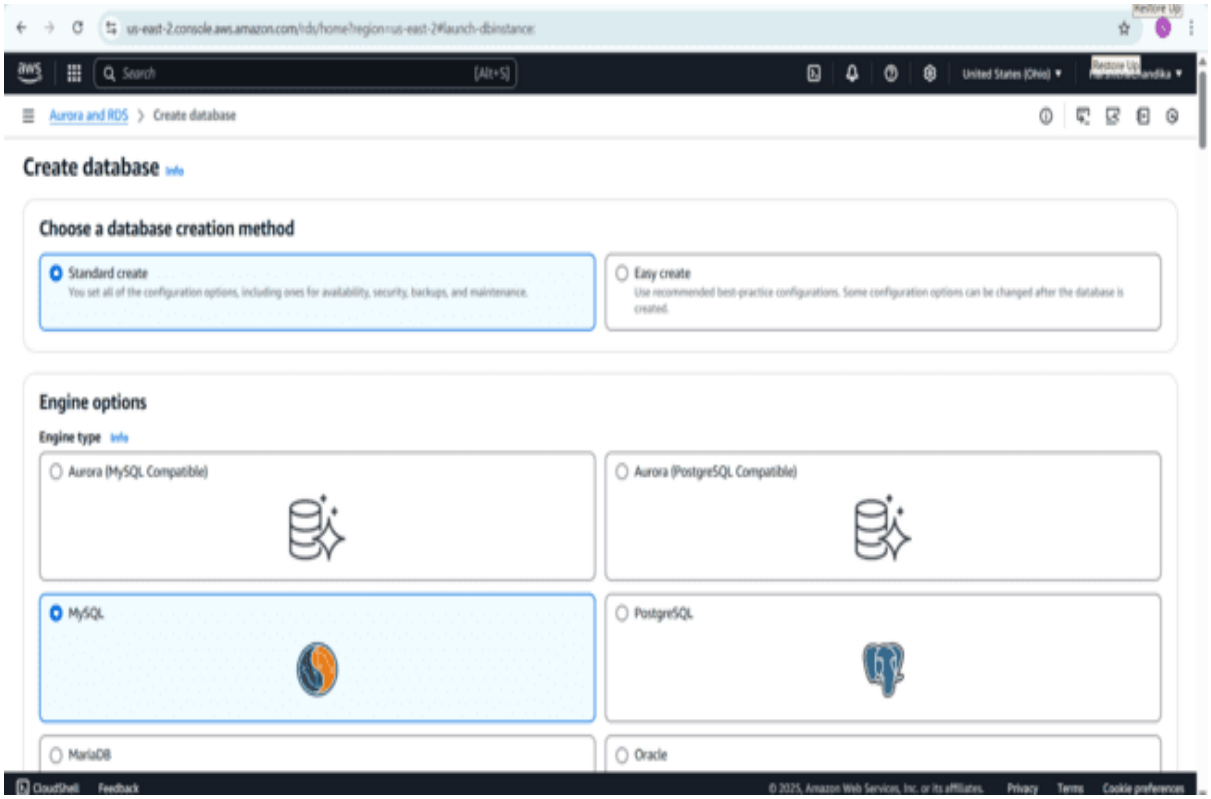


Fig 40: Navigate to RDS click on create database select standard create choose MySQL engine

Fig 41: Don’t disturb engine version of MySQL

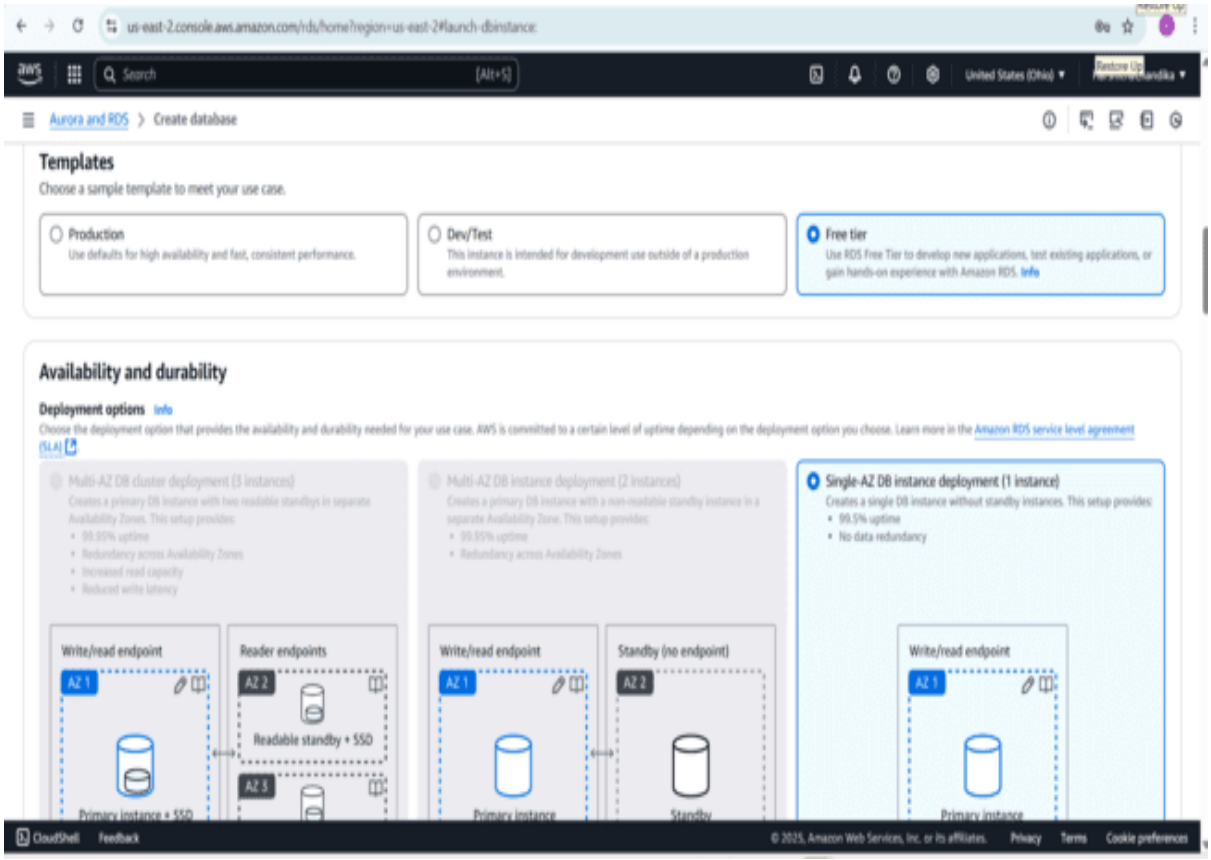


Fig 42: Go to templates select template as free tier and one single-AZ instance deployment will appear

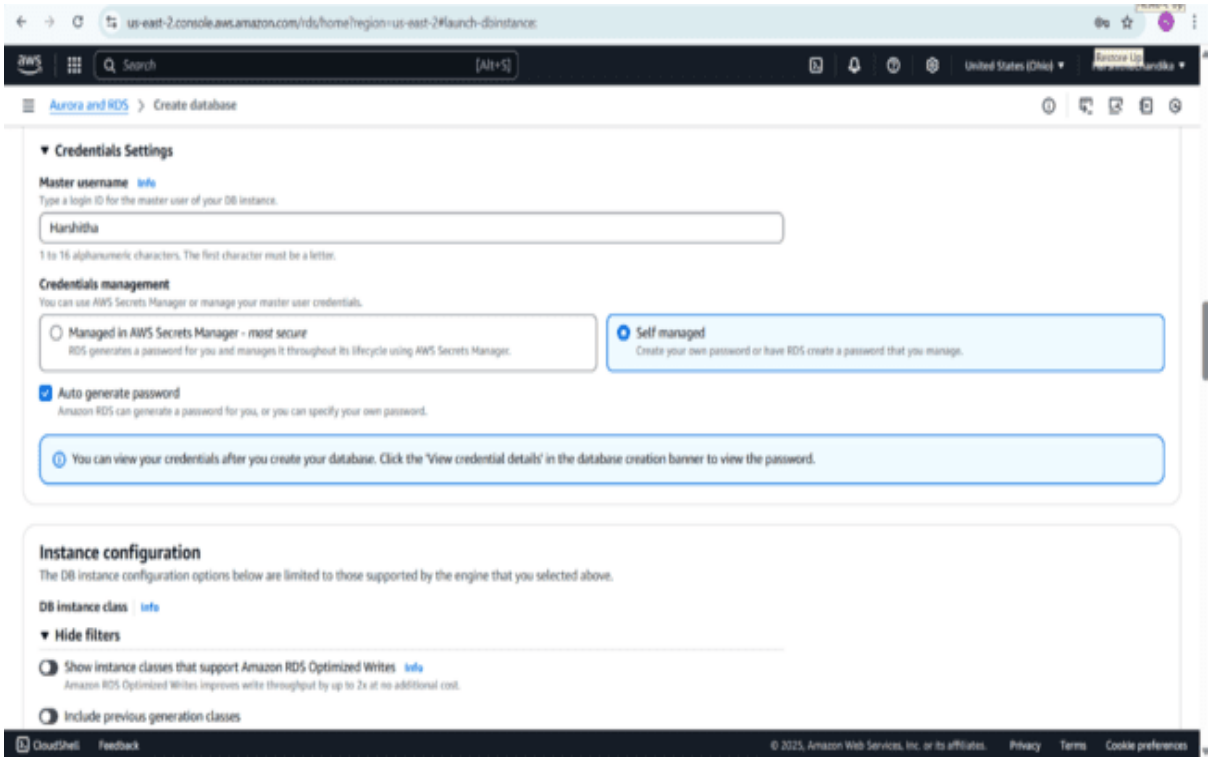


Fig 43

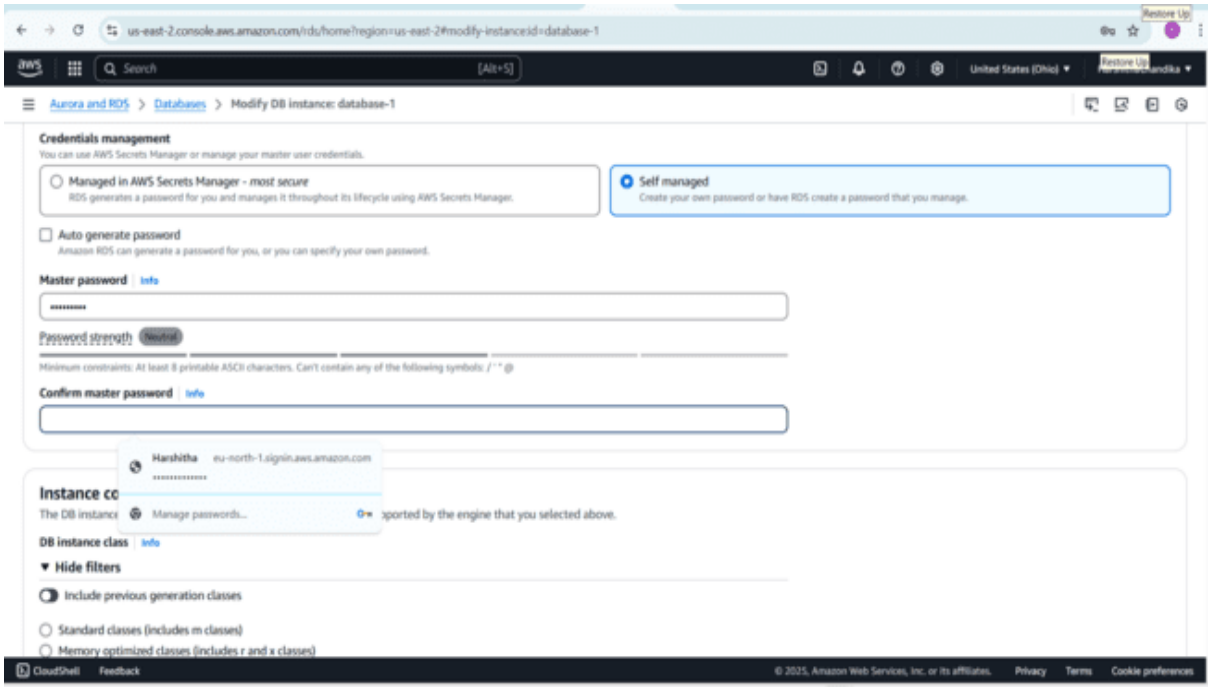


Fig 43&44: Give credentials self managed give master username and password

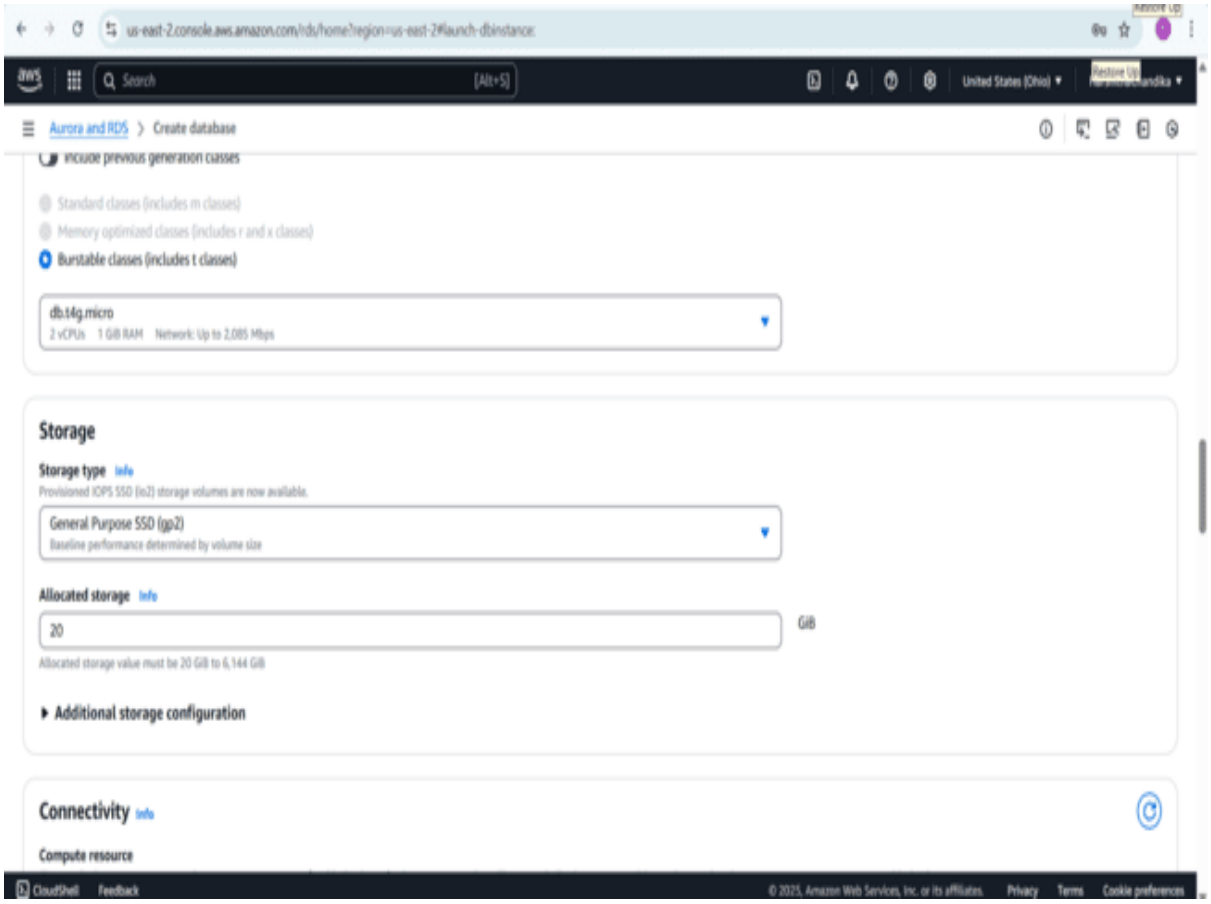


Fig 45: select db.t4g.micro

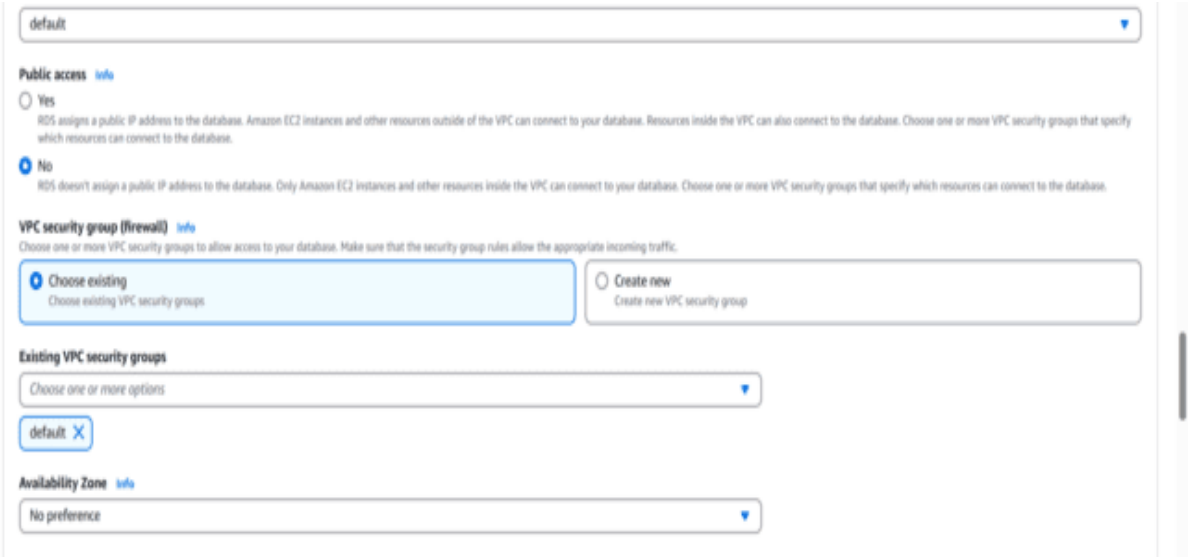


Fig 46: Select existing VPC

Fig 47: choose password authentication

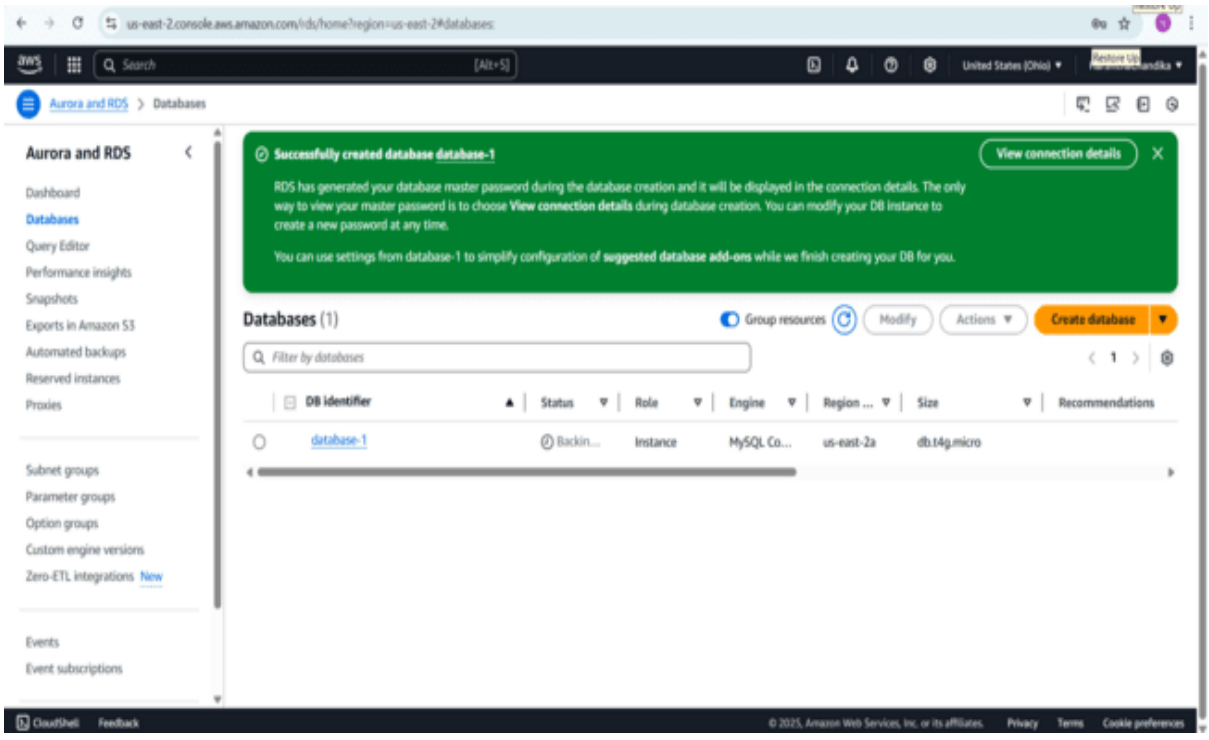


Fig 48: click on create database

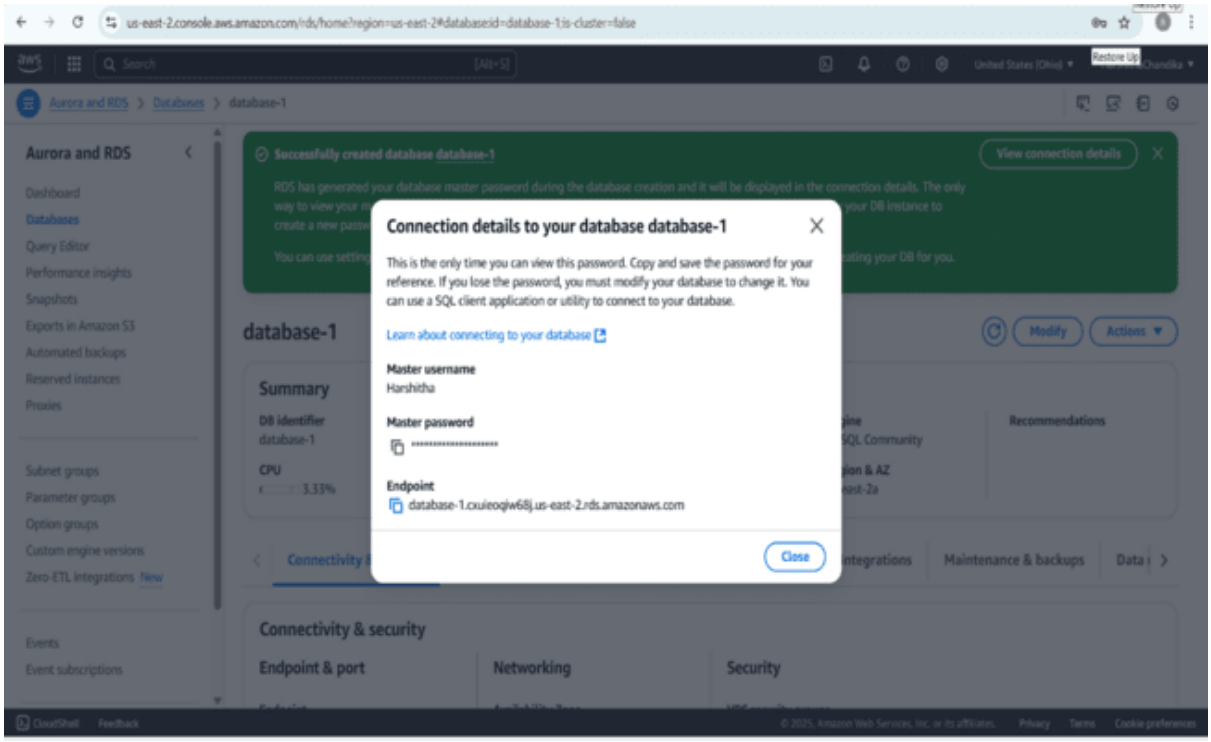


Fig 49: open the created database copy the password and endpoint

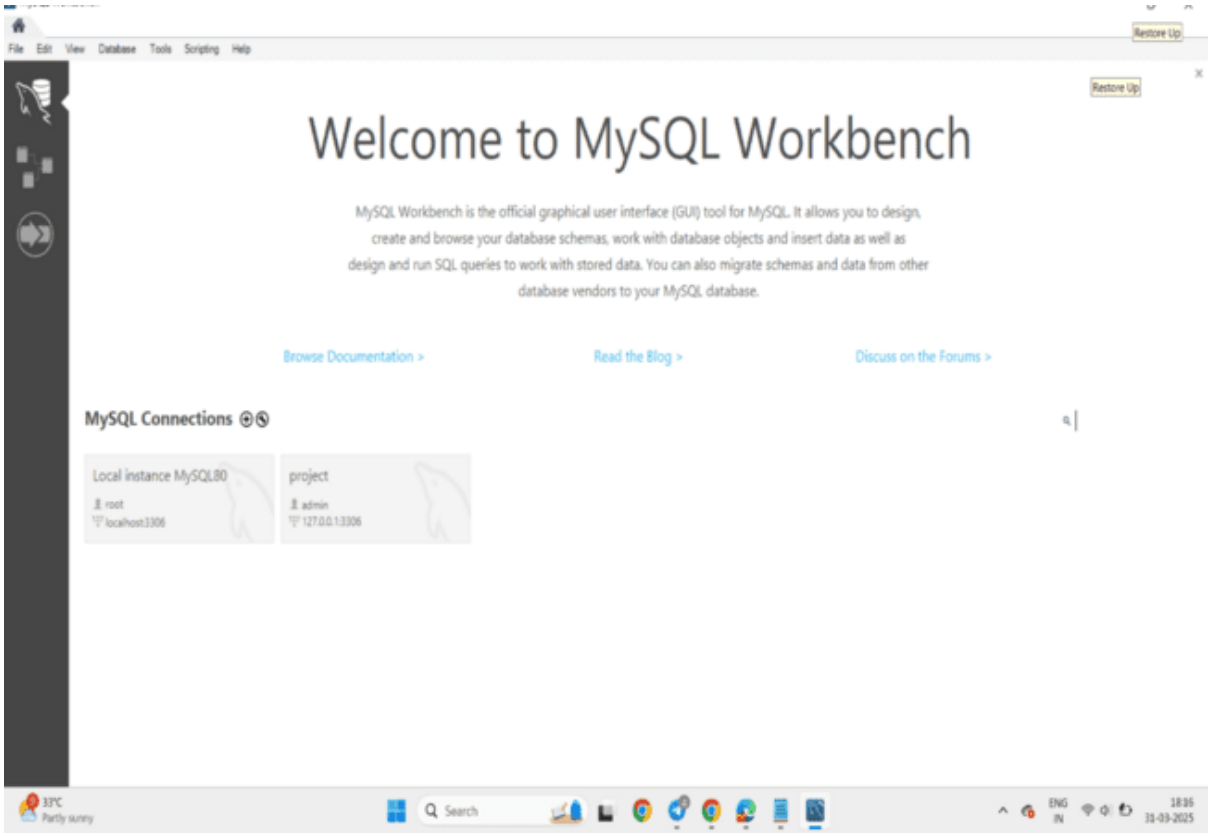


Fig 50: Now open MySQL workbench click on “+” icon

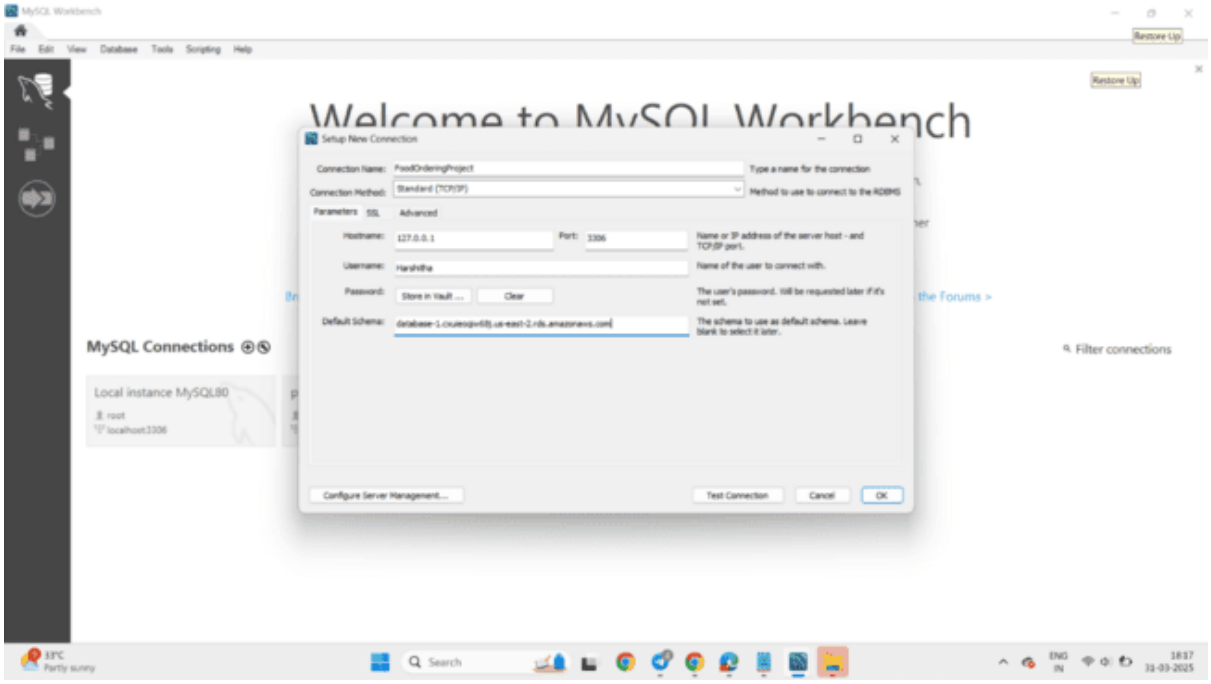


Fig 51: Give connection name , username and password will give according to RDS in place of default schema give endpoint which is copied from RDS database

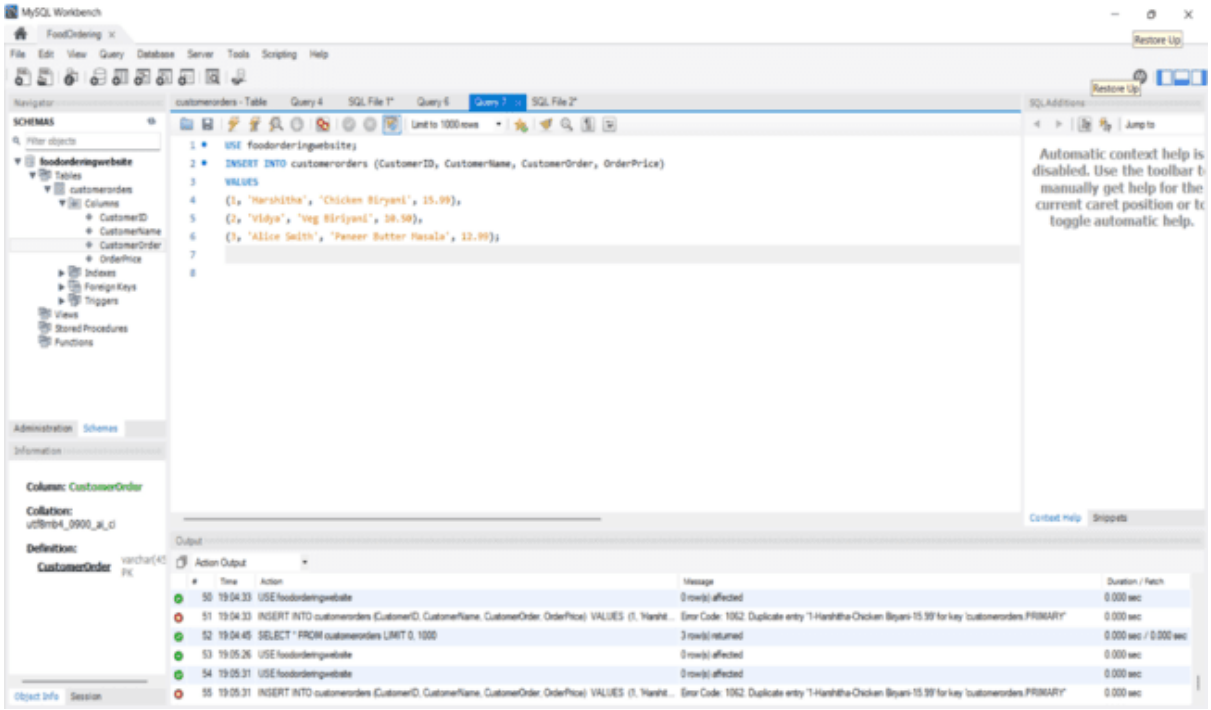


Fig 52: Now create a schema and create a table in schema give customer ID, name, order as columns

To create a database we have to run these commands in MySQL

```
CREATE DATABASE foodordering;
USE foodordering;
```

To create a table we have to run these commands in MySQL

```
CREATE TABLE customers (
    customer_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_name VARCHAR(100) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    phone_number VARCHAR(15) NOT NULL,
    address TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Now insert customer data into table

```
INSERT INTO customers (customer_ID,customer_name, ,
customer_order, customer_price)
VALUES
('1', 'Harshitha', 'chicken biriyani, '15.99'),
('2', 'vidya', 'veg biriyani', '10.50 '),
('3', 'Alice Smith', 'panner butter masala', '12.99');
```

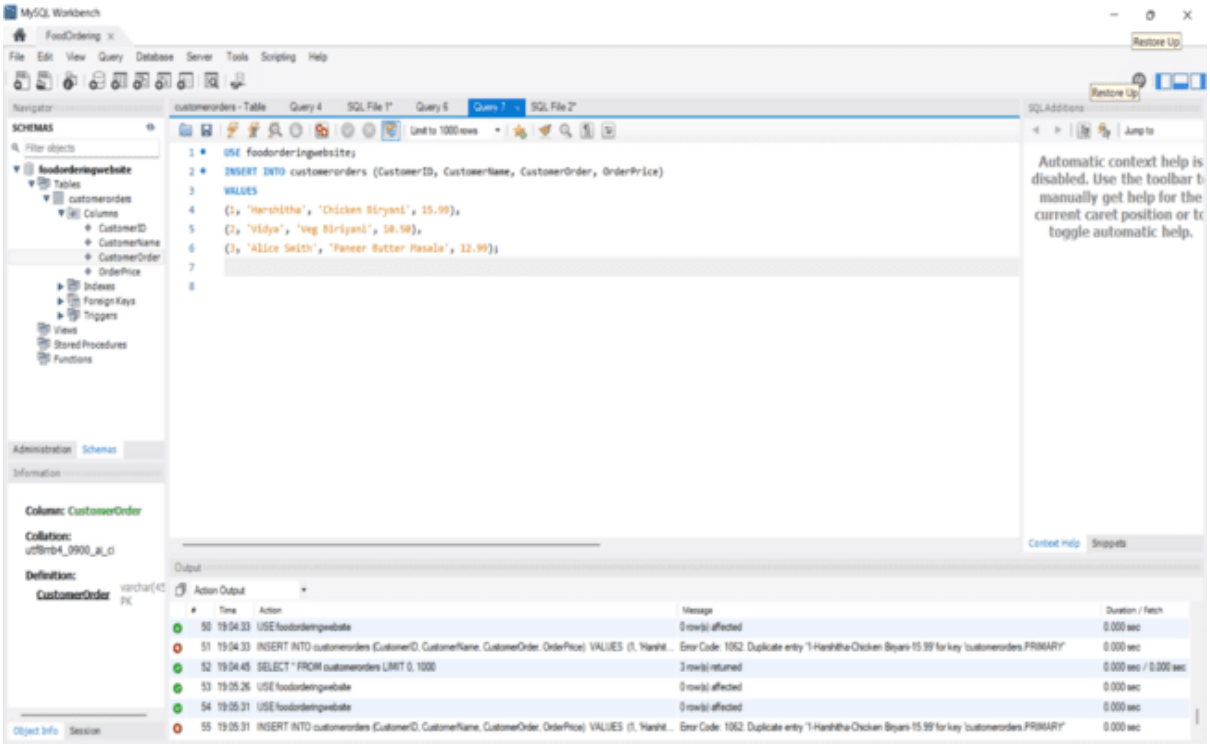


Fig 53: To select the table use the command "use foodordering website" Now insert the commands into the query like customer name, order, price

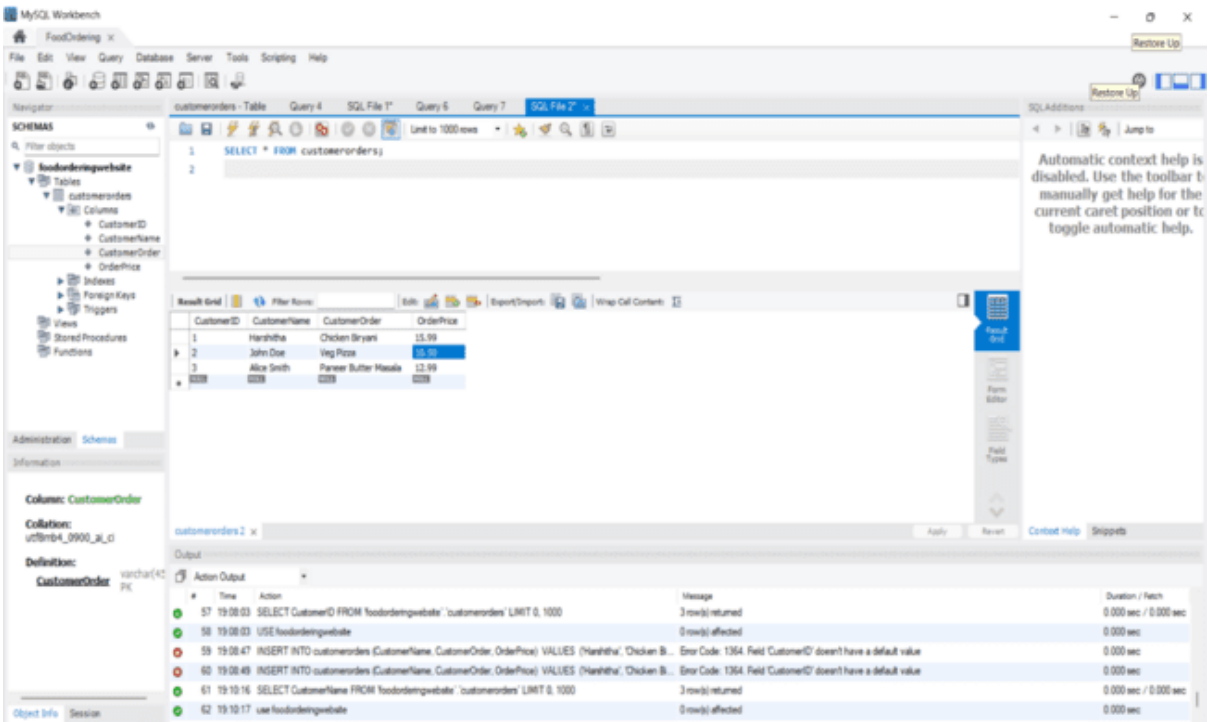


Fig 54: To get the data of the customers use the command "SELECT * FROM customerorders;" the table will appear

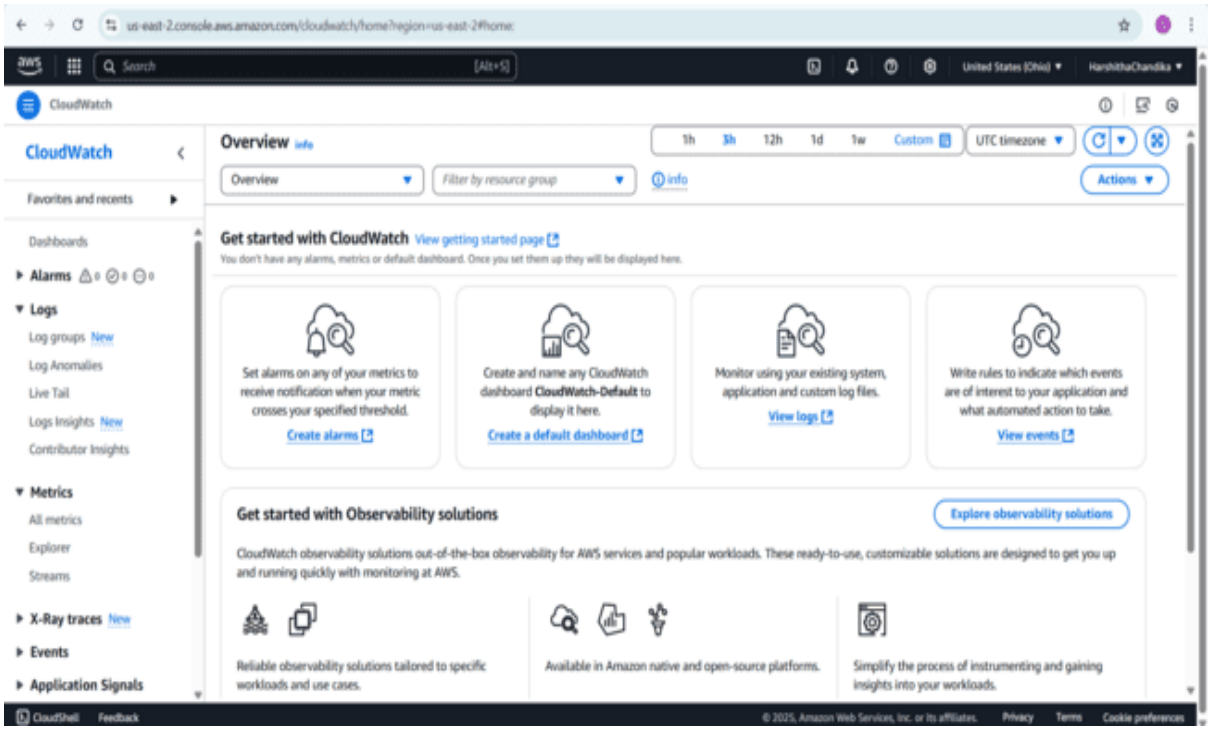


Fig 55: Navigate to CloudWatch. Click on logs dropdown

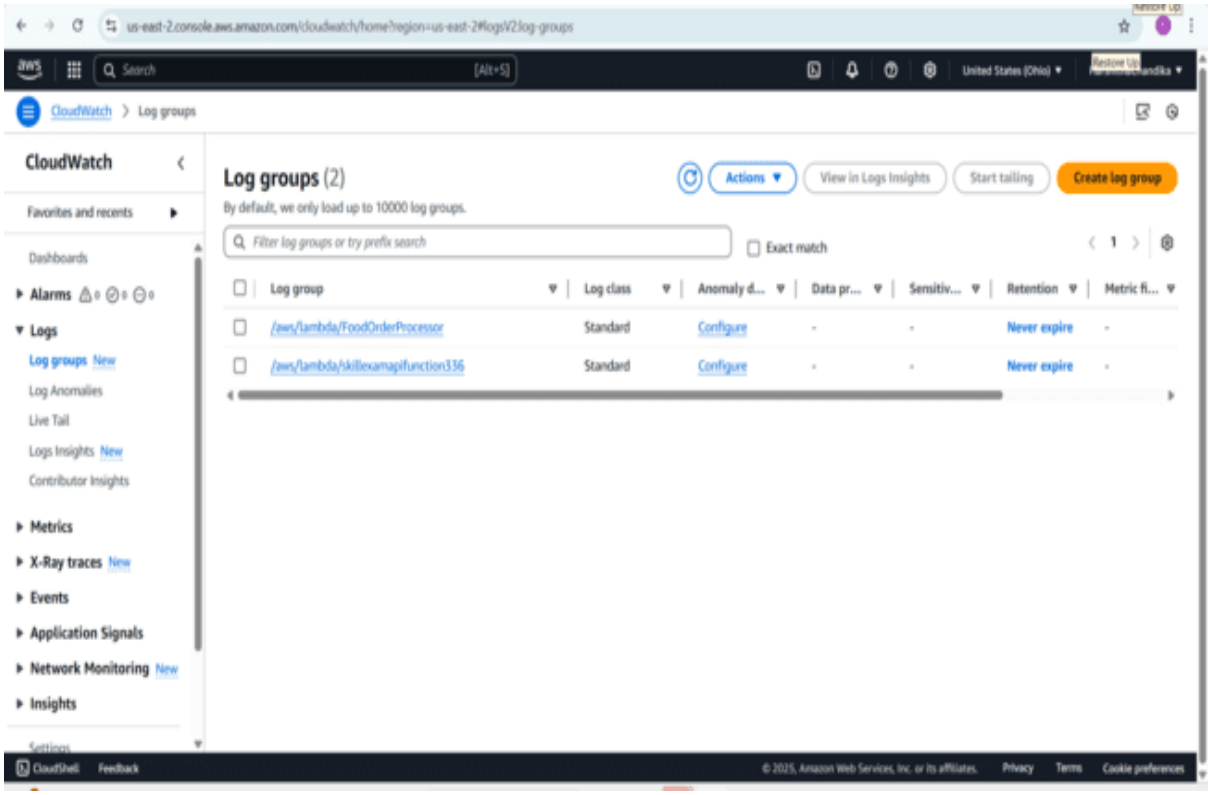


Fig 56: click on log groups we can see the created lambda function log group. Click on the lambda function log group

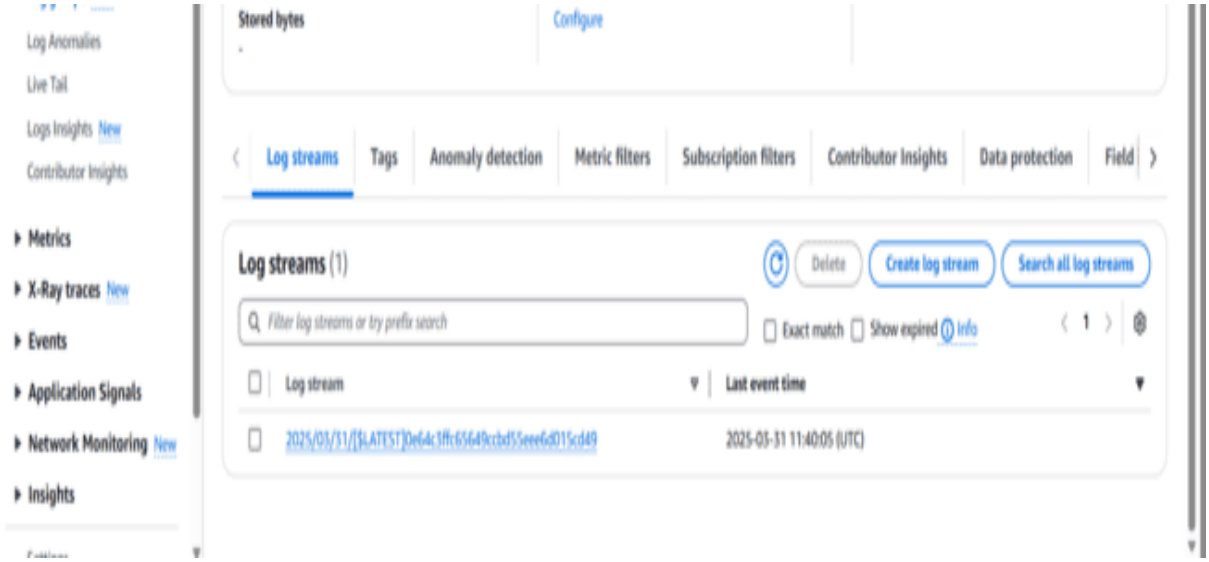


Fig 57: In Log streams we see the latest log stream of our lambda function click on it

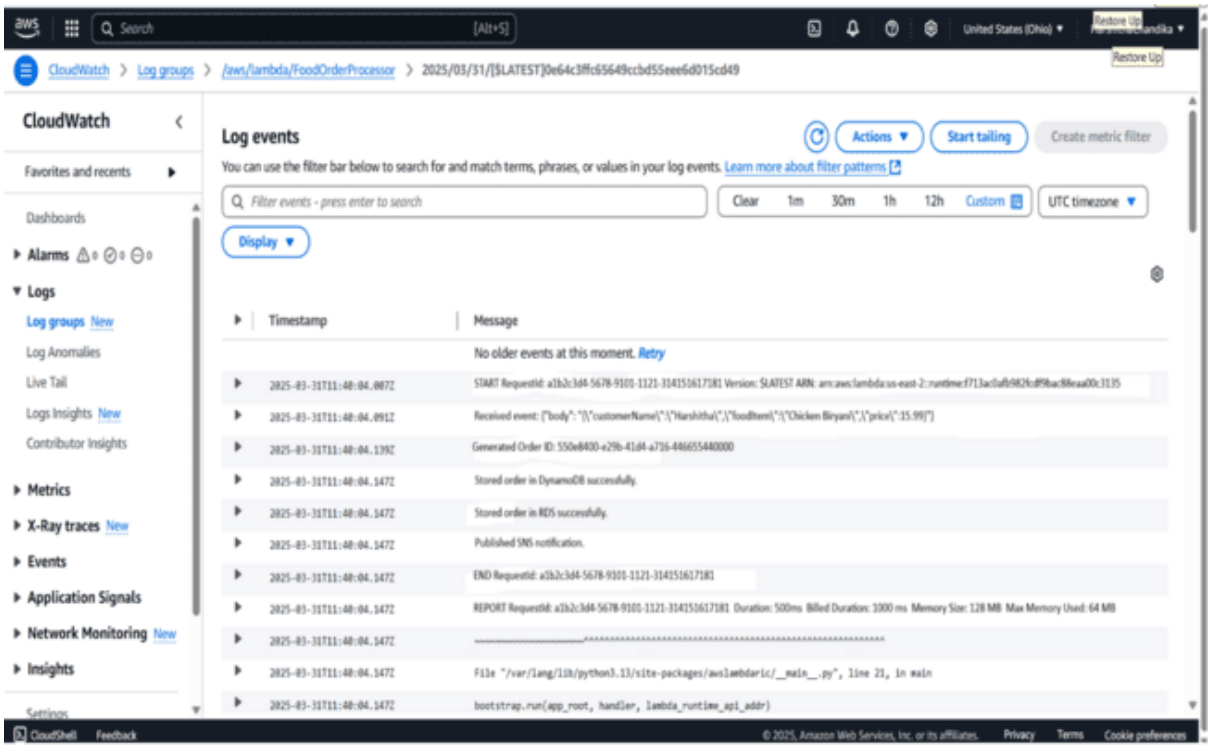


Fig 58: There we can see the customer id, name, order and price of the food in log events

Discussions & Results

The project successfully implemented a real-time digital food ordering system using AWS serverless services. The architecture leveraged Amazon API Gateway for handling client requests, AWS Lambda for backend processing, Amazon RDS and DynamoDB for storing structured and unstructured order data, and Amazon SNS for sending real-time notifications. This serverless setup ensured high scalability, reduced operational complexity, and cost-efficiency by using resources only when needed.

The system was tested by simulating user interactions—placing orders through a frontend or test events—and successfully recorded data across both RDS and DynamoDB. Each order triggered automated workflows, such as data storage and notification publishing, without the need for dedicated servers. The Lambda function generated unique order IDs, stored the order details in both databases, and published alerts to an SNS topic, confirming the smooth orchestration of services.

Incorporating user-specific preferences (e.g., favorite dishes or frequently ordered items) allowed basic personalization. Though not as advanced as a recommendation engine, the system could be easily extended to support personalized suggestions by integrating Amazon Personalize or leveraging user interaction history from DynamoDB.

Overall, the system demonstrated the effectiveness of AWS serverless architecture in building a responsive, scalable food ordering application. The results confirmed that such an approach is not only reliable but also adaptable, with opportunities for future enhancement such as advanced personalization, voice ordering integration, or AI-powered dynamic menus.

Conclusion

The Digital Food Ordering System built on AWS Serverless Architecture provides a fast, secure, and scalable solution for managing online food orders. By using services like S3, Lambda, API Gateway, DynamoDB, and SNS, the system ensures smooth user experiences, real-time updates, and reliable performance. It reduces infrastructure costs while improving efficiency, making it a modern and effective approach for food businesses in the digital life.

Future Work

Future improvements to the digital food ordering system can focus on enhancing personalization and scalability using AWS serverless services. Implementing hybrid recommendation models with user metadata and feedback can improve food suggestions. Automating model updates with AWS Lambda and Step Functions ensures real-time adaptability. Adding multi-language support and regional preferences makes the system more inclusive, while real-time dashboards and cross-domain suggestions can boost engagement and decision-making.

References

1. Amazon Web Services – Serverless Architectures with AWS Lambda.
   Retrievedfrom:
   https://docs.aws.amazon.com/lambda/latest/dg/welcome.html
2. Amazon Web Services – Building a Serverless Web Application.
   Retrievedfrom:
   https://aws.amazon.com/getting-started/hands-on/build-serverless-web-app-lambda-apigateway-s3-dynamodb-cognito/
3. Amazon Personalize – Real-time personalization and recommendations.
   Retrievedfrom:
   https://docs.aws.amazon.com/personalize/latest/dg/what-is-personalize.html
4. AWS Step Functions – Coordination of Serverless Workflows.
   Retrievedfrom:
   https://docs.aws.amazon.com/step-functions/latest/dg/welcome.html
5. Boto3 Documentation – AWS SDK for Python (Boto3).
   Retrievedfrom:
   https://boto3.amazonaws.com/v1/documentation/api/latest/index.html